

**T.C.**

**BANDIRMA ONYEDI EYLUL UNIVERSITY**

**FACULTY OF ENGINEERING AND NATURAL SCIENCES**

**DEPARTMENT OF COMPUTER ENGINEERING**

**BMI 3223 Software Engineering License Project**

**2025-2026 Spring Semester**

# ADEXOS - The Adaptive Exam Optimization System

**Ahmet Can Dedeağılı–2211504005**

**Mustafa Can–2211504074**

**Instructor:Assist. Prof. Erkut ARICAN**

**Bandırma,2026**

## TABLE OF CONTENTS

**SECTION 1: EXECUTIVE SUMMARY** ..... **4**
ADEXOS: The Adaptive Exam Optimization System ..... 4
**SECTION 2: PROJECT SCOPE & STAKEHOLDER ANALYSIS** ..... **5**
2.1 System Boundary Definition ..... 5
2.1.1 Functions not included in the application ..... 5
2.1.2 Features Not Included in the Project Scope ..... 6
2.2 Definition of Stakeholders ..... 7
2.2.1 Primary stakeholders : ..... 7
2.2.2 Secondary stakeholders : ..... 7
2.2.3 Tertiary stakeholders: ..... 7
2.3 Stakeholder Interest and Influence Matrix ..... 8
**SECTION 3: SOFTWARE PROCESS MODEL** ..... **10**
3.1 Selection of the SDLC Model ..... 10
3.2 Justification of the Selection ..... 10
**SECTION 4: SOFTWARE PROJECT MANAGEMENT PLAN** ..... **11**
4.1 Work Breakdown Structure ..... 11
4.2 Effort Estimate (Basic COCOMO I) ..... 12
4.3 CPM and Activity-on-Node (AoN) Analysis ..... 13
4.3.1.Activity on Node (AoN) ..... 14
4.3.2. Gantt Chart and Schedule Analysis ..... 15
4.3.3 Schedule Management and Brooks's Law ..... 15
4.4 Resource Plan and Strategic Constraints ..... 16
**SECTION 5: RISK MANAGEMENT** ..... **17**
5.1 Risk Identification ..... 17
5.2 Risk Matrix ..... 17
**SECTION 6: REQUIREMENTS ANALYSIS** ..... **21**
6.1 Functional Requirements - FR ..... 21
6.2 Non-Functional Requirements - NFR ..... 23
6.3 Use Case Diagram ..... 25
6.4 Use Case Descriptions ..... 25
UC-02: Read and Classify Text (OCR/NLP) (Detailed) ..... 25
UC-04: View Heatmap and Target Simulation (Detailed) ..... 27
UC-07: Edit Plan (Override) and Approve (Detailed) ..... 28
UC-09: Recalculate Route (Detailed) ..... 29
UC-10: Generate Mock Exam (Detailed) ..... 30
6.5 User Stories ..... 31
6.6 Requirements Traceability Matrix (RTM) ..... 31
**SECTION 7: SOFTWARE DESIGN** ..... **34**
7.1 Architectural Design ..... 34
7.2 Class Diagram ..... 36
7.3 Sequence Diagrams ..... 37
Sequence Diagram 1: Uploading and Classifying Incorrect Questions Using OCR (Ref.: UC1)

37
Sequence Diagram 2: Dynamic Route Calculation (Ref.: UC3)..... 38
7.4 Activity Diagram..... 40
7.5 State Machine Diagram..... 41
7.6 Component Diagram..... 43
Component Descriptions..... 44
7.7 Deployment Diagram..... 44
Node and Network Descriptions..... 46
7.8 Entity-Relationship(ER) Diagram..... 46
7.8.1 Database Table and Attribute Descriptions..... 48
1. STUDENT Table..... 48
2. ADVISOR Table..... 49
3. STUDY_PLAN Table..... 49
4. TASK Table..... 49
5. ERROR_QUESTION Table..... 50
6. MOCK_EXAM Table..... 50
7. BADGE Table..... 51
**SECTION 8: IMPLEMENTATION PLAN..... 51**
8.1 Technology Stack..... 51
8.1.1 Backend(Business Layer) : C# & ASP.NET Core..... 51
8.1.2 AI Microservice (AI Layer): Python, FastAPI, TensorFlow & LangChain..... 51
8.1.3 Database : MS SQL Server..... 52
8.1.4 Frontend (presentation layer): React.js..... 52
8.1.5 External services (APIs): Google Cloud Vision API & YÖK Atlas..... 52
8.2 Coding Standards..... 52
Naming Conventions..... 52
Version control strategy..... 53
8.3 Module Decomposition..... 53
8.4 Build and Deployment Pipeline (CI/CD Approach)..... 53
**SECTION 9: TESTING PLAN..... 54**
9.1 Test Strategy..... 54
9.2 Test Cases..... 55
9.2.1 Unit Testing..... 56
9.2.2 Integration Testing..... 58
9.2.3 System Testing..... 60
9.2.4 Acceptance Testing..... 62
9.3 RTM Verification..... 64
**SECTION 10 : MAINTANENCE PLAN..... 65**
10.1 Anticipated Maintenance Types..... 65
10.2 Change Management Process..... 66
10.3 Version Numbering Scheme..... 67
**SECTION 11: ETHICAL, LEGAL, AND SUSTAINABILITY CONSIDERATIONS..... 68**
11.1 Data Privacy and GDPR/KVKK Compliance..... 68
11.2 Ethical Implications..... 68

11.3 UN Sustainable Development Goals..... 69
**SECTION 12: CONCLUSION..... 69**
12.1 Summary of Accomplishments..... 69
12.2 Key Challenges Encountered.....69
12.3 Individual Reflections.....70
**SECTION 13: References.....71**

## SECTION 1: EXECUTIVE SUMMARY

### ADEXOS: The Adaptive Exam Optimization System

Competitive exams such as the university entrance exam (YKS), which shape the lives of millions of students, are currently a source of significant inequality in terms of opportunity and stress, due to rigid curricula and expensive tutoring and coaching systems. To pinpoint exactly which topic a student does not understand and create a dynamic learning program accordingly requires data analysis that pushes the limits of human capabilities.

ADEXOS is an autonomous AI-based educational navigation system designed to solve this problem at its root. The process begins when students use their smartphone camera to upload questions they cannot solve into the system; in the background, the data is then analyzed using optical character recognition and natural language processing technologies.

**Value Proposition:** ADEXOS's greatest innovation is that it renders static, impractical weekly schedules obsolete. The system functions like a GPS that calculates a new route for a vehicle stuck in traffic. If a student does not achieve 70% of their daily goals, the artificial intelligence activates at 11:59 p.m. and recalculates the remaining workload (rerouting) without disrupting the student's sleep or class schedule, then establishes an entirely new and achievable itinerary for the following day. Furthermore, thanks to the "teacher-advisor" approval mechanism (Human-in-the-Loop) integrated into the system, this process is subject to pedagogical oversight and does not operate entirely autonomously.

**Target group:** The platform's primary users are high school students preparing for the YKS, as well as teacher-advisors who monitor their progress. Our secondary target group consists of schools and educational institutions seeking to improve their success rates.

In summary, ADEXOS is not merely a collection of questions or a planning workbook, but a comprehensive educational coaching platform that emphasizes data, ensures equal opportunities (SDGs 4 and 10), and guides students directly toward their academic goals while safeguarding their mental health.

## SECTION 2: PROJECT SCOPE & STAKEHOLDER ANALYSIS

### 2.1 System Boundary Definition

ADEXOS's primary goal during the design phase was to optimize the learning processes of YKS students using a data-driven decision support system. However, to avoid diluting the technical focus and compromising the reliability of the machine learning models, our team clearly defined the scope of the system from the outset.

#### 2.1.1 Functions not included in the application

The following core features are to be developed as part of the platform:

- **OSYM prediction model based on historical data:** The system includes a model that analyzes the TYT/AYT questions from the past 10 years, as well as their difficulty levels and topic distribution. This allows for the creation of personalized, 40-question "practice exams" tailored to each student shortly before the actual exam.
- **Performance Analysis and Goal Simulation:** The system compares the student's test results with the current MEB/OSYM curriculum and adjusts them accordingly. It then calculates and visually displays the difference between the student's current performance level and the minimum score required by the target university (based on data from the YOK Atlas).
- **OCR-powered microanalysis (heat map):** Students can upload photos of questions they couldn't solve to the system. With the help of OCR and NLP modules running in the background, these questions are analyzed and broken down into subtopics. This allows for the creation of a heat map that not only identifies whether the student has gaps in major topics like "derivatives" but also pinpoints specific areas of weakness, such as "the multiplication rule for derivatives."

- **Dynamic route planning:** Instead of traditional, static programs, the system suggests a schedule that focuses on the subjects in which the student is struggling the most. If the student does not achieve 70% of their daily goals, the system recalculates the study plan—similar to a navigation device—and automatically updates the program for the next day (taking into account the remaining time and the student's sleep habits).
- **Tutor Approval:** The study plans generated by the artificial intelligence are not sent directly to the student. These plans are first sent as a "draft" to the tutor's dashboard. After the tutor has reviewed the plan and made any necessary changes, they make it available to the student.
- **Gamification:** To keep student motivation high, a rule-based badge system has been integrated into the system. Users can earn various badges, such as "Subject Ambassador," by achieving specific goals in sequence.

### 2.1.2 Features Not Included in the Project Scope

The main risks identified are scope creep and project failure. For this reason, the following items have been completely excluded from the project scope:

- **Live communication and private lessons:** ADEXOS is an asynchronous platform that does not operate in real time. To avoid unnecessary expansion of the server infrastructure, features such as live classes, video conferencing, or instant messaging have not been integrated into the project.
- **Content creation and e-commerce:** The system is not designed as a question bank or a video learning platform, but as a diagnostic tool. For this reason, the integration of a payment infrastructure, the sale of subscriptions, or the creation of question content were excluded from the scope of the project from the outset.
- **Social features:** To avoid any legal risks related to the requirements of the KVKK and GDPR, social features such as general leaderboards, where students compare their scores with one another, or adding friends are not included in the system. The project is entirely focused on individual development.
- **Reading open-ended questions:** The OCR and NLP models used in the system are designed to read exclusively multiple-choice questions and standardized mathematical formulas. The evaluation of handwritten essays or open-ended answers is not currently planned**.**

- **Psychological support:** The system provides support based exclusively on grades and academic data. Developing a chatbot capable of offering advice to alleviate exam anxiety or stress among students was deemed to fall outside the project's legal and ethical framework.
- **Hardware integration (IoT):** Data entry into the system is performed exclusively via the web interface or the smartphone camera. No hardware integration with smart pens or optical scanners is planned.

### 2.2 Definition of Stakeholders

During the system design phase, all groups affected by the project were identified as follows:

#### 2.2.1 Primary stakeholders :

- **Students:** They are at the heart of the system; they constitute the main user group, responsible for scanning questions, entering test results, and implementing the plans developed by artificial intelligence.
- **Advisors/coaches:** educators who monitor students' academic progress and review and approve the draft plans generated by the system.
- **System administrators and data team:** the technical team responsible for ensuring system security, updating databases, and verifying the accuracy of AI models.

#### 2.2.2 Secondary stakeholders :

- **Parents:** Even though they do not directly use the app, this group wants to track students' academic progress and funds the system's use**.**
- **Educational institutions:** Schools and tutoring centers that may jointly use the platform for their students to improve their institution's success rate.
- **Cybersecurity auditors:** These are experts responsible for verifying that sensitive student data collected in the system (target universities, test scores, etc.) is protected by security standards such as AES-256.

#### 2.2.3 Tertiary stakeholders:

- **MEB & ÖSYM:** Although they do not interact directly with the platform, they are the project's most critical external dependencies, as changes they make to the curriculum or exam standards have a direct impact on the system's database structure.

- **Cloud and API service providers:** external providers such as AWS, Azure, or Google Vision OCR, which ensure the proper functioning of the system and its underlying infrastructure.
- **Target universities and software communities:** external factors that have an indirect impact on the project's potential outcomes.

### 2.3 Stakeholder Interest and Influence Matrix

In order to effectively coordinate the efforts and interests of the project's stakeholders, the following strategy was adopted:

| **Stakeholder Category** | **Influence** | **Interest** | **Management Strategy** | **Why Do We Think This Way? (Justification)** |
|---|---|---|---|---|
| **Advisor** | HIGH | HIGH | Manage Closely | The implementation of the system in the field depends on the agreement of the teachers involved. Since any errors made by the artificial intelligence are highly likely to lead to the system being rejected, their feedback is of great importance to the project. |
| **System administrators and data experts** | HIGH | HIGH | Manage Closely | This team is fully responsible for the performance of the artificial intelligence, the system's responsiveness, and database integration. |

| | | | | |
|---|---|---|---|---|
| **Cybersecurity Auditors** | HIGH | LOW | Maintain Satisfaction | Even if they are not interested in the project's business model, security standards must be strict, as any security flaw they might discover in the system could bring the project to a standstill. |
| **MEB & OSYM** | HIGH | LOW | Satisfy / Adapt (Keep Satisfied) | Even though they are not directly involved in the project, they are responsible for the curriculum. The system must adapt immediately to any changes. |
| **Students** | LOW | HIGH | Keeping them informed | Since their success depends on the system, they have a strong interest in it, but they have no influence over architectural decisions. The goal is to motivate them through a user-friendly and seamless interface. |
| **Parents** | LOW | HIGH | Keep Informed | You're not interested in the technical details, you care only about the results. Our goal is to provide you with |

| | | | | |
|---|---|---|---|---|
| | | | | clear and reliable analytical charts. |
| **Cloud and API service providers** | LOW | LOW | Monitoring | They provide infrastructure services only. You only need to monitor bills, quotas, and occasional outages. |

*Table 1: Stakeholder Interest and Influence Matrix*

**SECTION 3: SOFTWARE PROCESS MODEL**

## 3.1 Selection of the SDLC Model

Our team chose the Agile methodology and the Scrum framework as the software development life cycle (SDLC) for the ADEXOS platform. Since the project focuses on artificial intelligence and requires continuous feedback throughout the process, using rigid, one-way models like Waterfall would have been risky. Therefore, we opted for a more iterative and flexible approach that can respond quickly to changes during the development process.

### 3.2 Justification of the Selection

The team's decision to adopt the Agile methodology is based on three main reasons directly related to the project's technical structure, as explained in Section 2:

- **Need for dynamic feedback:** Due to the system's architecture, study plans generated by the AI are not sent directly to students; they are first reviewed and approved by subject teachers. This is where Scrum's short development cycles, known as sprints, and sprint review meetings come into play: this enables constructive feedback from teachers to be integrated into the code and algorithms much more quickly.
- **Technical risks associated with AI models:** Components such as the "OSYM prediction engine" and "OCR-supported microanalysis" are not features that can be created flawlessly from the start using traditional development methods. For structures this complex, the iterative development process provided by the Agile

approach offers a decisive advantage: models are trained in small chunks and tested at every stage, which ensures that accuracy rates are gradually improved.

- **Flexibility regarding external dependencies:** the project's two most critical external dependencies are the Ministry of National Education (MEB) and the Student Selection and Placement Center (ÖSYM). Any changes made by these institutions to the curriculum or exam system require immediate updates to the system's database. This is precisely where Agile's adaptable structure comes into play: thanks to the rapid updating of the Product Backlog, these external changes can be managed without disrupting the development process.




*Figure 1: ADEXOS Agile Sprint Cycle and Development Phases*

## SECTION 4: SOFTWARE PROJECT MANAGEMENT PLAN

### 4.1 Work Breakdown Structure

A three-level hierarchical project structure plan has been established; it covers the entire scope of the ADEXOS project and also includes the necessary integration and documentation efforts.

- **1.0 ADEXOS System**
  - **1.1 Planning and Requirements Analysis**

    1.1.1 Stakeholder interviews and development of the functional requirements specification (SRS)

    1.1.2 Development of the project schedule and risk matrix

  - **1.2 System Design**

    1.2.1 Database and data dictionary design

    1.2.2 Architecture and interface design

  - **1.3 Implementation**

    1.3.1 Integration of the C# ASP.NET database

    1.3.2 Development of an NLP question analysis module (Python)

    1.3.3 Dynamic AI-powered route calculation and forecasting engine

    1.3.4 User roles and gamification module

  - **1.4 Testing and Verification (Verification & Validation)**

    1.4.1 Unit testing and module integration testing

    1.4.2 Accuracy testing for OCR algorithms

  - **1.5 Deployment**

    1.5.1 Server installation and cloud integration

    1.5.2 User documentation and handover

### 4.2 Effort Estimate (Basic COCOMO I)

Since the system includes both web infrastructure components and machine learning components, it has been classified as “semi-detached” due to its complexity. The estimated number of lines of code for the project is 15 KLOC. (Semi-detached model constants: a=3.0, b=1.12, c=2.5, d=0.35).

- **Rough effort calculation :**
  E = 3.0 * ( 15 ) ^ 1.12 ~ 61.38 people - months
- **Integration and Contingency:**
  In accordance with software planning principles, a contingency allowance equivalent to 15% of the gross cost has been included to account for unforeseeable errors that may arise during module integration.
- *Total workload = 61.38 * 1.15 ~ 70.58 people - months*
- **Development Time :**
  D = 2.5 * ( 70.58 ) ^ 0.35 ~ 10.93 months
- **Average Team Size :**
  N = E/D = 70.58/10.93 ~ 6.45 people (A full-time team of six people has been planned for the project.)

### 4.3 CPM and Activity-on-Node (AoN) Analysis

The Critical Path was calculated using the Forward/Backward Pass method after analyzing task dependencies. (Durations are in “weeks”).

| ID | Task | Duration | Predecessor | ES | EF | LS | LF | Float |
|---|---|---|---|---|---|---|---|---|
| A | Requirements Analysis | 2 | - | 0 | 2 | 0 | 2 | 0 |
| B | System Design | 3 | A | 2 | 5 | 2 | 5 | 0 |
| C | Database & API Setup | 3 | B | 5 | 8 | 7 | 10 | 2 |
| D | AI & OCR Models | 5 | B | 5 | 10 | 5 | 10 | 0 |
| E | Frontend & UI Dev | 3 | B | 5 | 8 | 7 | 10 | 2 |

| ID | Task | Duration | Predecessor | ES | EF | LS | LF | Float |
|---|---|---|---|---|---|---|---|---|
| F | Integration & Testing | 3 | C, D, E | 10 | 13 | 10 | 13 | 0 |
| G | Deployment | 1 | F | 13 | 14 | 13 | 14 | 0 |

*Table 2: Critical Path Method (CPM) and Task Analysis Data*

**Critical path:** a sequence of tasks with a float of "0," which determines the minimum duration of the project.

- Critical path: A -> B -> D -> F -> G

### 4.3.1.Activity on Node (AoN)

To enable the project team to better track the CPM (Critical Path Method) data calculated in the table above and to clarify the logical dependencies between tasks, the following Activity-on-Node (AoN) network diagram was created.

Each node in this diagram, designed in accordance with systems engineering standards, clearly indicates the early start (ES), early finish (EF), late start (LS), and late finish (LF) dates of the corresponding task, as well as its float.

When examining the diagram, the critical path—which has a flexibility margin of "0" and directly determines the minimum project completion time—is particularly highlighted in the network diagram (with thick red arrows). Normal tasks (C and E), which have a float—that is, a tolerance for delay—have, on the other hand, been integrated into the main sequence in such a way as not to compromise the critical path.

The "Activity on Node" analysis is explained as follows :

| ES | ID(duration) | EF |
|---|---|---|
| LS | FLOAT | LF |

*Table 3: Activity-on-Node (AoN) Legend Structure*

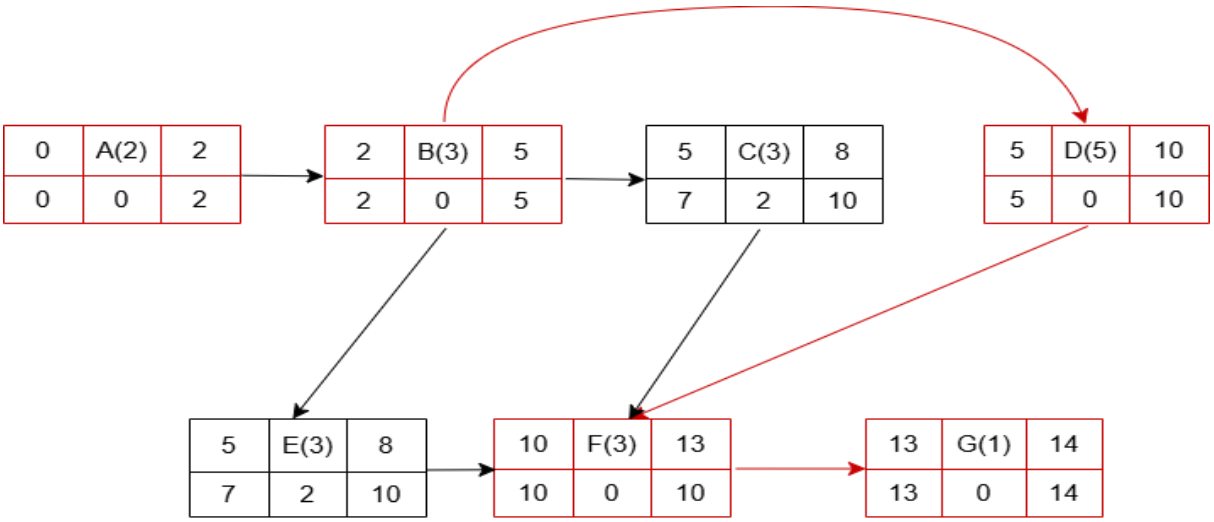



*Figure 2: Activity-on-Node (AoN) Network Diagram with Critical Path Analysis*

### 4.3.2. Gantt Chart and Schedule Analysis

The 14-week project development cycle was represented using the Gantt chart below to track the distribution of work packages across the schedule.

In the chart, tasks on the critical path are highlighted in red based on data from the CPM analysis. This clearly shows which tasks, if delayed, would directly impact the project's final delivery date (W14).

To address the limitation of traditional Gantt charts, which "cannot represent flexibility buffers," buffer values have been incorporated into the chart. The gray blocks added to the end dates of non-critical tasks (C and E) represent the time flexibility that can be utilized for these tasks without compromising the project. This is intended to enable advance planning for bottlenecks in resource allocation and parallel work.

| | A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | ID | | W1 | W2 | W3 | W4 | W5 | W6 | W7 | W8 | W9 | W10 | W11 | W12 | W13 | W14 |
| 2 | A | Requirements Analysis | | | | | | | | | | | | | | |
| 3 | B | System Design | | | | | | | | | | | | | | |
| 4 | C | Database & API Setup | | | | | | | | | | | | | | |
| 5 | D | AI & OCR Model | | | | | | | | | | | | | | |
| 6 | E | Frontend & UI Dev | | | | | | | | | | | | | | |
| 7 | F | Integration & Testing | | | | | | | | | | | | | | |
| 8 | G | Deployment | | | | | | | | | | | | | | |

*Figure 3: Project Development Gantt Chart with Critical Path*

### 4.3.3 Schedule Management and Brooks's Law

In the event of any delay during the project timeline, the principle that "person-months are not interchangeable" has been adopted. As defined by Frederick Brooks **[2]**, adding new personnel to a late software project will only delay it further, as it increases the communication overhead ( n( n- 1)/ 2 ) and the ramp-up time of new members will consume the existing team's productive hours. Therefore, in case of potential bottlenecks on the Critical Path (Task D), strategies such as scope reduction or extending the delivery date will be pursued instead of hiring new staff.

## 4.4 Resource Plan and Strategic Constraints

The distribution of project resources across WBS work packages has been designed under specific strategic constraints to balance the communication overhead and apply the "Resource Leveling" principle:

- **Project Manager and Scrum Master (Assigned WBS: 1.1, 1.5):** Responsible for the process and risk management of the project. Restricts team expansion to exactly 2 members to keep communication costs under control. Acts as the strict enforcer of the aforementioned "Brooks's Law" **[2]** by prioritizing scope management over team expansion during schedule deviations.
- **Requirements Analyst and UX Designer (Assigned WBS: 1.1, 1.2):** Conducts stakeholder interaction and interface analysis. The primary constraint is to secure final draft approval from the guide teachers before the Implementation phase begins, thereby blocking "Scope Creep," which is one of the project's most significant risks **[4]**.
- **Backend Developer - C# (Assigned WBS: 1.2, 1.3.1, 1.3.4):** Undertakes the construction of the core architecture and API modules. Must ensure that the developed data processing modules are ready for the Integration phase (1.4.1)—the main intersection point of the Critical Path—with zero scheduling tolerance **[3]**.
- **Artificial Intelligence (AI) Specialist (Assigned WBS: 1.3.2, 1.3.3):** Develops the prediction engine and OCR systems. Because this role is located on the project's Critical Path (Task D), no concurrent (parallel) secondary tasks can be assigned until
- these autonomous processes are completed. This constraint is the strictest rule of the project plan in order to prevent over-allocation.

- **Frontend Developer (Assigned WBS: 1.3.4, 1.5):** Develops the client side and gamification screens. Progresses asynchronously with the backend team. In case of potential interface bottlenecks, utilizes the "2-week Float" of Task E as a strategic buffer to avoid delaying the master schedule.
- **QA (Quality Assurance) Engineer (Assigned WBS: 1.4):** Manages quality assurance and testing processes. Prepares validation scenarios (test cases) concurrently with the coding phase, without waiting for the development process to conclude, thereby ensuring the early detection of defects **[1]**.

## SECTION 5: RISK MANAGEMENT

To ensure the security of the ADEXOS development lifecycle, a comprehensive risk management plan has been developed. This process includes the steps of risk identification, qualitative analysis, and the development of proactive response strategies.

### 5.1 Risk Identification

The potential risks of the ADEXOS project were identified by the project team using brainstorming and FMEA (Failure Mode and Effects Analysis) techniques based on the technical architecture. Taking into account the project's YKS-oriented AI integration, external dependencies (OSYM/MEB), and stakeholder dynamics, 10 fundamental risks were identified.

### 5.2 Risk Matrix

The identified risks were represented in the color-coded 2D matrix below based on their probability of occurrence (Probability: 1–5) and their impact on the project (Impact: 1–5).

*(Risk IDs are listed in the matrix. Color scale: 🟩 Low [1–4], 🟨 Medium [5–10], 🟧 High [12–16], 🟥 Critical [20–25])*

| Probability | | | | | | |
|---|---|---|---|---|---|---|
| | 5(Very High) | 5 | 10 | 15 | 20 | 25 |
| | 4(High) | 4 | 8 | 12 (R1, R5) | 16 | 20 |
| | 3(Medium) | 3 | 6 | 9 (R7) | 12 (R3) | 15 (R4) |
| | 2(Low) | 2 (R10) | 4 (R9) | 6 (R6) | 8 | 10 (R2) |
| | 1(Very Low) | 1 | 2 | 3 | 4 | 5 (R8) |
| | | 1(Very Low) | 2(Low) | 3(Medium) | 4(High) | 5(Very High) |
| | | Impact | | | | |

*Figure 4: Qualitative Risk Assessment Matrix*

### 5.3 Risk Register Table

The categories, scores (Probability × Impact), and mitigation strategies for the 10 identified risks are detailed in the Risk Register Table below:

| ID | Risk Description | Category | Probability | Impact | Risk Score | Response Strategy | Contingency Plan |
|---|---|---|---|---|---|---|---|
| R1 | OCR Model Errors: The OCR/NLP module misreads complex math formulas or charts, generating inaccurate heat maps. | Technical / AI | 4 | 3 | 12 | Mitigate | Implement a feedback interface allowing users to manually crop or input the misread question. |
| R2 | Curriculum / Exam Format Change: Sudden changes in the national exam (ÖSYM/MEB) format or topic distribution during development. | External Dependency | 2 | 5 | 10 | Accept | Utilize the flexibility of the Agile process to implement rapid Sprint updates to the database schema and AI training sets. |
| R3 | Guide Teacher Rejection: Teachers find the AI-generated plans untrustworthy (overriding them frequently), slowing down field adoption. | Stakeholder / UX | 3 | 4 | 12 | Mitigate | Integrate short "Explainable AI" tooltips detailing why specific study routes were generated to build trust. |
| R4 | Critical Path (Task D) Bottleneck: The AI Specialist spends more time than planned on machine learning training, causing schedule delays. | Management / Schedule | 3 | 5 | 15 | Mitigate | Enforce Brooks's Law (no new personnel added); instead, reduce the scope of the gamification module (Task 1.3.4). |
| R5 | C# and Python Integration Mismatch: JSON parsing or timeout errors occur during API data exchange between the backend and the NLP module. | Technical / Architecture | 4 | 3 | 12 | Avoid | Define strict API contracts using Swagger/OpenAPI at the beginning of the Sprint and set up preemptive Mock tests. |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| R6 | Scope Creep: Stakeholders or team members request new features (e.g., live classes, open-ended question evaluation) as the project progresses. | Scope Management | 2 | 3 | 6 | Avoid | Strictly enforce constraints by immediately moving unauthorized new requests to the "Out-of-Scope" list once the SRS is approved. |
| R7 | Server Crash / Performance Degradation: Simultaneous mock exam simulations crash the server during peak times (April/May before the exam). | Performance / Infrastructure | 3 | 3 | 9 | Transfer | Outsource server hosting to cloud providers (e.g., AWS/Azure) and configure automatic Auto-Scaling rules. |
| R8 | Data Breach / Privacy Violation: Sensitive student data (target universities, exam scores) is exposed due to a cyberattack. | Security | 1 | 5 | 5 | Avoid | Utilize AES-256 encryption in the database and completely exclude high-risk social features (e.g., public leaderboards) from the system. |
| R9 | Gamification Fatigue: The rule-based badge mechanism loses its motivational impact and becomes tedious for students over time. | Stakeholder / UX | 2 | 2 | 4 | Accept | Do not alter the core code architecture in the current phase; plan dynamic weekly quests for the subsequent release (Product Backlog). |
| R10 | Version Control / Merge Conflicts: Team members experience Git/GitHub merge conflicts, causing localized effort loss. | Technical / Management | 2 | 1 | 2 | Mitigate | Establish strict branching strategies (e.g., Feature Branching) at the start of each Sprint and enforce daily "Pull" rules to maintain high synchronization. |

*Table 4: Project Risk Register and Mitigation Strategies*

## SECTION 6: REQUIREMENTS ANALYSIS

This section addresses the prerequisites that will form the foundation for the architectural design and development processes of the ADEXOS platform. Each item is detailed in accordance with software engineering standards, while adhering to the system boundaries (In-Scope / Out-of-Scope) outlined in Section 2 and the work packages planned in Section 4.

### 6.1 Functional Requirements - FR

The core tasks that the system is expected to perform directly through user interactions or background processes are listed below:

- **FR-01 (Image Processing with OCR):** The system must be able to process photos of multiple-choice questions uploaded by students and, using its integrated OCR (Optical Character Recognition) module, convert the text and standard mathematical formulas in the image into a digital format. Handwritten or open-ended questions, however, should not be accepted by the system, and the user should be notified accordingly.
- **FR-02 (NLP-Based Subtopic Classification):** The Python NLP module within the system should evaluate the question—which has been converted into digital text—using Natural Language Processing algorithms, match it with the specific subtopic it pertains to (e.g., "Product Rule in Calculus") based on the current MEB/ÖSYM curriculum tree, and save it to the database.
- **FR-03 (Heat Map - Heat Map Generation):** The system should provide a Heat Map interface based on a red-to-green color scale that visualizes the student's academic weaknesses at a micro level by cross-analyzing the incorrect question records accumulated in the user's database and the entered practice test scores.
- **FR-04 (Target-Delta Simulation Calculation):** The system should retrieve the current YÖK Atlas minimum score data for the target university and department specified in the student's profile, calculate the difference (delta) between this target score and the student's current average on practice exams, and display this information on the screen both numerically and graphically.
- **FR-05 (Creating a Dynamic Plan Draft):** The AI engine should prioritize the student's weakest topics, as identified in their Heat Map, and generate a draft of a

weekly adaptive study schedule (sprint) based on the user's sleep and school hours, which have been predefined in the system.

- **FR-06 (Counselor Approval and Override Workflow):** Study plans generated by artificial intelligence should not be sent directly to the student; instead, they should be sent as drafts to the relevant guidance teacher's dashboard. The teacher should be able to manually make changes (overrides) to this draft on a per-course and per-class basis and approve the plan in its final form.
- **FR-07 (Student Task Panel Synchronization):** Students should be able to view only the study plans approved by their guidance counselor as "Daily Tasks" on their interface and mark completed tasks without any issues.
- **FR-08 (Smart Route Recalculation):** The system should check the student's daily task completion rate at the end of each day. If this rate falls below 70%, the system should automatically reschedule the student's schedule for the following day based on the remaining days of the week and current study hours, much like a navigation device recalculating a route.
- **FR-09 (Generating Practice Exams Based on Past Data):** The system should be able to generate personalized 40-question "Practice Exam" templates for each student using a machine learning model that examines and analyzes the difficulty levels and topic distributions of TYT/AYT questions from the past 10 years (with C# backend integration).
- **FR-10 (Rule-Based Gamification Module):** The system should track students' consistency in completing tasks on the platform; it should automatically award virtual motivation badges—such as "Discipline Ambassador"—to users who achieve specific goals consecutively (for example, those who complete all their tasks for seven consecutive days).
- **FR-11 (Administrator and User Role Management):** System administrators should be able to add new mentor teachers and students to the platform, grant the necessary permissions, and manually update the master database (Data Dictionary) to reflect any changes made to the MEB/ÖSYM curriculum.

### 6.2 Non-Functional Requirements - NFR

Non-functional requirements that define the system's infrastructure standards—such as quality, security, performance, and usability—are addressed in detail below to ensure the robustness of the software architecture. These requirements have been developed to be fully aligned with the legal obligations outlined in Section 2 (KVKK/GDPR) and the system design objectives in Section 4.

- **NFR-01 (Performance and Scalability - Performance):** The ADEXOS platform must have a scalable cloud infrastructure capable of handling sudden traffic spikes that may occur during weekends when practice exams are concentrated across Turkey and as the YKS approaches. The system must be able to manage at least 10,000 concurrent active student sessions without any loss of performance; page load and response times on the interface must be limited to a maximum of 2 seconds.
- **NFR-02 (Security and Data Privacy - Security):** Given the sensitivity of the data processed in the field of educational technology, it is essential that the platform be protected by the highest standards of cybersecurity. Student test scores, target university information, heat map analyses, and passwords must be stored in the database (Data at Rest) using AES-256 encryption; during data transmission (Data in Transit), the TLS 1.3 protocol must be used. In addition, the system must be fully compliant with the OWASP Top 10 security principles to protect against threats such as SQL Injection, XSS, and data leaks. A "zero legal risk" policy under the KVKK/GDPR should also be adopted, adhering to the principle of not including social comparison features such as leaderboards.
- **NFR-03 (Usability and Human-Computer Interaction - Usability):** Given that the target audience will be spending long hours in front of the screen conducting practice tests and studying, the user interface must include full support for a Dark Mode to minimize eye strain. The navigation architecture must be designed based on the "3-Click Rule" principle, ensuring that both students and instructors can access the main modules (Heat Map, Practice Test, Task Panel) with no more than three clicks. Since the platform includes a photo-taking feature, full compatibility with mobile browsers (Responsive Design) is also a mandatory requirement.
- **NFR-04 (Availability and Uninterrupted Operation - Availability):** Since ADEXOS is a platform that students use according to their own schedules

(asynchronously) rather than for real-time live classes, it must be accessible 24/7. The system must guarantee 99.9% uptime. Intensive tasks such as server maintenance, database backups, and large-scale data updates for OCR/NLP models should be performed between 2:00 AM and 5:00 AM, when traffic volume is at its lowest.

- **NFR-05 (Sustainability and Flexibility - Maintainability):**Given the possibility of sudden changes to the curriculum or exam systems (such as question distributions, adding or removing topics, etc.) by the Ministry of National Education (MEB) and the Student Selection and Placement Center (ÖSYM)—which are the project's most critical external dependencies—the system's database architecture (Data Dictionary) must be designed to be highly modular. In the event of a curriculum update, the system must be able to adapt to the new curriculum solely through database tables, without any intervention in the core artificial intelligence code (AI Engine).
- **NFR-06 (Artificial Intelligence: Fault Tolerance and Reliability):** The accuracy rate of the OCR reading and NLP-based question tagging modules, which form the foundation of the system, must be maintained at a minimum of 92%. When faced with images it cannot read or that are of insufficient quality (blurry, excessively dark, etc.), the algorithm should protect data integrity by sending the user a clear error message ("The image is not clear; please take another photo in adequate lighting") rather than causing the system to crash or produce incorrect classifications (hallucinations).

## 6.3 Use Case Diagram




*Figure 5: ADEXOS Platform Use Case Diagram*

## 6.4 Use Case Descriptions

### UC-01: Upload Question Photo (Brief)

- **Actor:** Student
- **Description:** The student uploads a clear photo of the multiple-choice question they answered incorrectly via the interface, thereby submitting the image to the system for OCR processing.

**UC-02: Read and Classify Text (OCR/NLP) (Detailed)**

- **Use Case Name:** Read and Classify Text (OCR/NLP)
- **Actors:** ADEXOS AI Engine, Student
- **Pre-conditions:** The student has logged in to the platform securely and successfully uploaded a clear photo of a multiple-choice question via the interface.
- **Main Flow:**
    - The system retrieves the uploaded image file and saves it to a temporary cloud bucket.
    - The OCR module scans the image to extract textual data and standard mathematical formulas.
    - The NLP module processes the extracted text to analyze the context of the question.
    - The AI Engine compares this context with the current MEB/ÖSYM curriculum database.
    - The system tags the question with a specific subtopic label, such as “Product Rule in Derivatives,” rather than a general category like “Mathematics.”
    - The classified question data is permanently saved to the student’s personal error database.
- **Alternative Flows:**
    - **Blurry or Unreadable Image:** If the OCR module cannot extract meaningful text from the image due to low resolution, glare, or insufficient lighting, the system terminates the classification process, deletes the temporary file, and displays the following error message on the interface: “The image is not clear enough for analysis. Please take another photo under adequate lighting.”
    - **Handwriting Detection:** If the system detects handwritten text or an open-ended question that falls outside its scope, it rejects the upload and informs the user that only standard multiple-choice formats are supported.
- **Post-conditions:** The question is successfully digitized, categorized, and added to the student's data pool, triggering an automatic update of the learning map.

**UC-03: Complete Daily Tasks (Brief)**

- **Actor:** Student

- **Description:** The student marks the tasks assigned to them on the board as “Completed”; this action updates the progress metrics and feeds data into the Gamification module.

**UC-04: View Heatmap and Target Simulation (Detailed)**

- **Use Case Name:** View Heatmap and Target Simulation
- **Actor:** Student
- **Pre-conditions:** The student must have previously specified a target university and department in their profile, completed at least one task, or submitted an incorrect answer.
- **Main Flow:**
    - The student goes to the “Performance Analysis” module.
    - The system retrieves the student's processed error logs and practice test results from the database.
    - The system generates a Heat Map interface that visualizes mastery of subtopics using a color scale ranging from red to green.
    - Simultaneously, the system retrieves the current cutoff score for the student’s target university from the YÖK Atlas dataset.
    - The system calculates the difference (Delta) between the student’s current estimated score and the target cutoff score.
    - The interface visualizes the Delta value via a simulation graph to show how close the student is to their target.
- **Alternative Flows:**
    - **Missing Target University Data:** If the student has not yet selected a target university, the simulation graph will not be displayed; instead, the following prompt will appear on the interface: “To view the performance simulation, please select a target university from your profile.”
- **Post-conditions:** Students can successfully analyze their academic performance through a comprehensive, data-driven visualization without having to interpret raw numerical data on their own.

**UC-05: Monitor Student Progress (Brief)**

- **Actor:** Advisor

- **Description:** Advisor monitors the task completion rates, heat maps, and overall academic performance of the students under their supervision via a read-only dashboard.

**UC-06: Approve Study Plan (Brief)**

- **Actor:** Advisor
- **Description:** Advisor reviews the draft study plan generated by the AI and approves it to activate the plan; this makes the plan visible on the student's interface.

**UC-07: Edit Plan (Override) and Approve (Detailed)**

- **Use Case Name:** Edit Plan (Override) and Approve
- **Actor:** Advisor
- **Pre-conditions:** The ADEXOS AI Engine has completed the weekly analysis for a student and generated a draft study plan (sprint). Advisor has logged into their dashboard.
- **Main Flow:**
    - Advisor goes to the "Pending Approvals" tab on the dashboard.
    - They open the AI-generated draft plan for a specific student and review the proposed schedule alongside the student's current Heat Map data.
    - They notice that an educational adjustment is needed (for example, the AI may have allocated too many hours to Physics, ignoring the student's fatigue from recent exams).
    - They click the "Edit Plan (Override)" button.
    - They reduce the Physics study block by 1 hour and add a "Rest/Reading" block in its place.
    - They click the "Confirm and Publish" button.
- **Alternative Flows:**
    - **Direct Approval (Without Overriding):** If the guidance counselor reviews the draft plan and finds it well-balanced and educationally appropriate, they can skip the editing phase entirely and click the "Approve" button directly.
- **Post-conditions:** The status of the study plan is updated in the database from "Draft" to "Active," and the plan appears immediately on the student's personal task board.

**UC-08: Generate Draft Plan (Brief)**

- **Actor:** ADEXOS AI Engine (System Actor)
- **Description:** The AI Engine automatically generates a balanced draft weekly study schedule by analyzing the gaps in the student's heat map and their biological time constraints (sleep/school hours).

**UC-09: Recalculate Route (Detailed)**

- **Use Case Name:** Recalculate Route
- **Actor:** ADEXOS AI Engine
- **Pre-conditions:** The system clock reaches 11:59 PM, and there is an active study plan containing daily assignments that has been approved by the student's advisor.
- **Main Flow:**
    - The system checks the student's task completion status for that day by querying the database.
    - It determines the completion percentage by calculating the ratio of completed tasks to the total number of tasks.
    - The algorithm detects that the completion rate has fallen below the critical threshold of 70%.
    - The AI Engine selects the uncompleted tasks and adds them back to the active priority queue.
    - The system queries the student's predefined biological time constraints (sleep and school hours) and the remaining days of the week.
    - The algorithm dynamically redistributes the workload to make up for lost progress without exceeding daily maximum work hour limits, and creates an optimized new schedule for the following day.
- **Alternative Flows:**
    - **Threshold Met or Exceeded:** If the student has successfully completed 70% or more of their daily tasks, the system does not make any structural changes to the schedule for the following days and directly updates the counter in the Gamification module.
- **Post-conditions:** The student's dashboard is updated with an adaptive to-do list—generated in a manner similar to how a GPS recalculates a route—to reflect the schedule for the following day.

**UC-10: Generate Mock Exam (Detailed)**

- **Use Case Name:** Generate Mock Exam
- **Actors:** Student, ADEXOS AI Engine
- **Pre-conditions:** Students request a practice exam through the student interface, and the system contains ÖSYM question datasets from the past 10 years.
- **Main Flow:**
    - The student clicks the "Create Practice Exam" button on the dashboard.
    - The AI Engine accesses TYT/AYT question data from the past 10 years (including difficulty coefficients and topic distribution frequencies).
    - The system queries the current Heat Map to identify the student's weakest subtopics.
    - The algorithm selects exactly 40 questions, giving slightly more weight to the student's weak areas while strictly adhering to the actual ÖSYM topic distribution standards.
    - The system compiles these questions into a personalized digital exam format.
    - The student is presented with a personalized "Practice Exam" interface.
- **Alternative Flows:**
    - **Insufficient Student Data:** If a new student requests a practice exam without having generated sufficient Heat Map data, the AI will disable the personalized weakness weighting and generate a standard practice exam based solely on historical ÖSYM frequency data.
- **Post-conditions:** A 40-question practice exam is successfully created; after it is completed, the results are fed back into the Heat Map and Target Simulation modules.

**UC-11: Manage Users and Curriculum (Brief)**

- **Actor:** System Admin
- **Description:** The system administrator performs manual updates to the MEB/ÖSYM core database to create accounts, configure authorization settings, and reflect changes to the official curriculum.

### 6.5 User Stories

In this section, the requirements to be added to the Product Backlog during the ADEXOS platform development process have been defined in the User Story format ("As a [role], I want to [action] in order to [benefit]") in accordance with Agile methodology standards.

- **US-01 (Student Perspective):** As a **student**, I want to upload photos of the questions I answered incorrectly to the system so that I can automatically and accurately identify which subtopics I need to work on without getting lost in the vast question banks.
- **US-02 (Advisor Perspective):** As a **guidance counselor**, I want to review the draft lesson plans generated by artificial intelligence and make manual adjustments (overrides) to them; this way, I can balance the workload by incorporating my educational experience and intuition before the plan is presented to the student.
- **US-03 (Student Perspective):** As a **student**, I want to view my academic performance through a color-coded heat map interface so that I can instantly and visually identify the areas where I'm falling short, rather than trying to interpret complex numerical data.
- **US-04 (Student Perspective):** As a **student**, I want the system to automatically recalculate my study plan whenever my daily task completion rate drops below 70%; that way, I can easily adjust to the schedule the next day without feeling guilty or wasting time on manual planning.
- **US-05 (System Admin Perspective):** As a system administrator, I want to monitor the accuracy rates of the ÖSYM prediction engine and OCR module through log reports so that I can continuously improve the algorithm by monitoring the performance of the machine learning model.

## 6.6 Requirements Traceability Matrix (RTM)

This matrix provides a comprehensive traceability map that links each Functional Requirement (FR) and core Non-Functional Requirement (NFR) to the relevant design use cases and validation test cases. This structure prevents scope creep during the implementation phase while maintaining full test coverage.

| Req ID | Requirement Name & Description | Type | Priority | Related Use Case | Test Case ID (Section 9) |
|---|---|---|---|---|---|
| **FR-01** | **OCR Image Processing:** Extraction of text and mathematical formulas from uploaded images of questions. | Functional | High | UC-01: Upload Question Photo | TC-01, TC-02, TC-19 |
| **FR-02** | **NLP Sub-Topic Classification:** Processing the text and tagging it with MEB/ÖSYM subtopics. | Functional | High | UC-02: Read and Classify Text | TC-03 |
| **FR-03** | **Visual Heatmap Generation:** Visualization of color-coded academic performance analysis. | Functional | Medium | UC-04: View Heatmap and Simulation | TC-04 |
| **FR-04** | **Target-Delta Simulation:** Calculation of score differences using official YÖK Atlas data. | Functional | Medium | UC-04: View Heatmap and Simulation | TC-05, TC-18, TC-20 |
| **FR-05** | **AI Dynamic Sprint Planning:** Creating weekly draft schedules based on the student's areas of weakness. | Functional | High | UC-08: Generate Draft Plan | TC-06 |
| **FR-06** | **Advisor Review & Override:** Allowing guidance counselors to | Functional | High | UC-06: Approve Study Plan / | TC-07, TC-08 |

| | | | | | |
|---|---|---|---|---|---|
| | manually edit and approve draft plans. | | | UC-07: Edit Plan | |
| **FR-07** | **Task Board Synchronization:** Matching assignments approved by the guidance counselor directly with the student interface. | Functional | High | UC-03: Complete Daily Tasks | TC-09 |
| **FR-08** | **Smart Route Recalculation:** When the completion rate falls below 70%, the schedule for the coming days should be reoptimized. | Functional | High | UC-09: Recalculate Route | TC-10, TC-11 |
| **FR-09** | **OSYM Historical Mock Exam:** A collection of 40-question personalized practice tests. | Functional | Medium | UC-10: Generate Mock Exam | TC-12 |
| **FR-10** | **Gamification Module:** Defining automatic milestone badges (e.g., the Continuity Series). | Functional | Low | UC-03: Complete Daily Tasks | TC-13, TC-17 |
| **FR-11** | **Curriculum & Admin Management:** Enabling manual database updates for curriculum changes. | Functional | Medium | UC-11: Manage Users and Curriculum | TC-14 |

| | | | | | |
|---|---|---|---|---|---|
| **NFR-01** | **System Scalability:** Managing 10,000 concurrent sessions with a response time of less than 2 seconds. | Non-Functional | High | Entire Platform Architecture | TC-15 |
| **NFR-02** | **Data Encryption:** BRequiring AES-256 encryption for data at rest and TLS 1.3 encryption for data in transit. | Non-Functional | High | Database Enclave | TC-16 |

*Table 5: Requirements Traceability Matrix (RTM)*

## SECTION 7: SOFTWARE DESIGN

In this section, the system, structural, and behavioral architecture of the ADEXOS platform has been modeled in accordance with UML standards and software development design principles.

### 7.1 Architectural Design

The ADEXOS project was designed using a hybrid architectural approach (N-tier client-server and microservices) to ensure the system's longevity and scalability, as well as seamless interaction between different technologies (C# and Python). To adhere to the "separation of concerns" principle from software development, the system was divided into modular layers.

The system's overall architecture consists of the following fundamental layers, external services, and technological protocols:

- **Presentation Layer:** this is the access point where user interactions (students and lead teachers) take place. It includes the web dashboard and mobile user interface (UI) components. The **HTTPS (TLS 1.3)** protocol is used to ensure secure communication with the business layer.

- **Business Logic Layer:** This is the central application server where business logic is executed and where data flow and permissions are managed. It is built on the **C# ASP.NET Core** infrastructure.
- **AI (Artificial Intelligence) Microservices Layer:** An independent microservice based on **Python** that handles system tasks requiring significant computing power (routing, mock exam generation). In accordance with the component and deployment architecture, it communicates with the main C# server via **REST interfaces based on gRPC and JSON** over a high-speed intranet (internal LAN).
- **Data Access Layer:** This is the **SQL Server** relational database layer where the system's logical data model is stored. To maximize security, it is locked against any external access and communicates securely with the C# backend via **TCP/IP port 1433**..
- **External APIs (external services):** The platform uses the Google Cloud Vision API as an external dependency to offload optical character recognition (OCR), as well as YÖK Atlas data services for calculating target point simulations (Delta), and accesses these services via secure HTTPS tunnels.

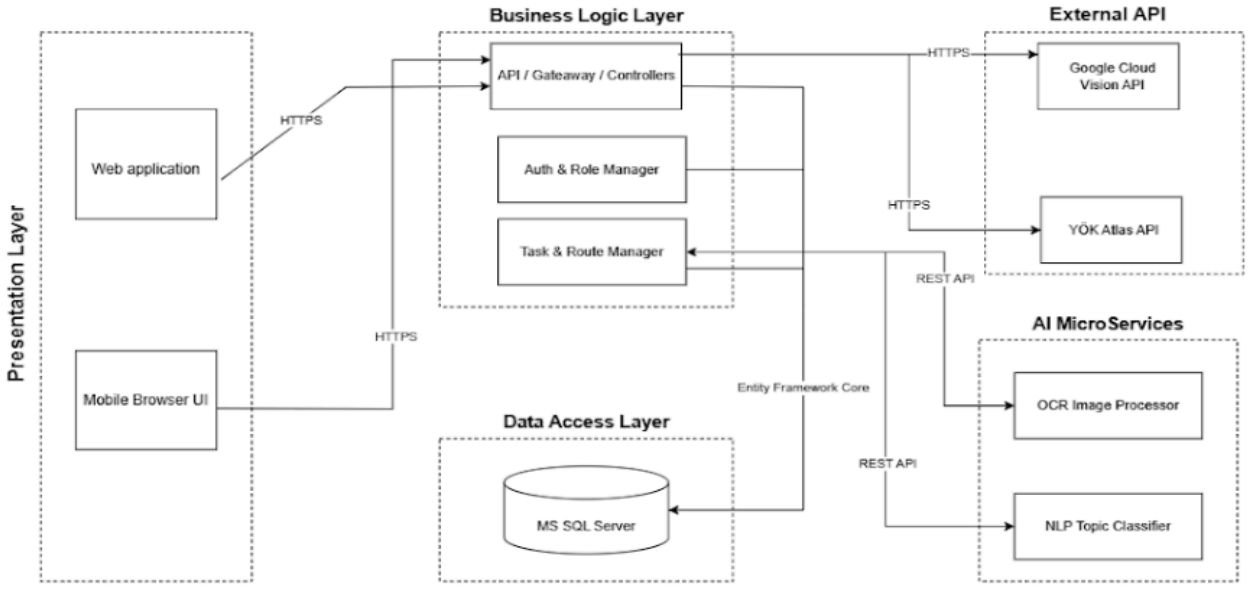



*Figure 6: ADEXOS Platform Hybrid Architectural Design Diagram*

### 7.2 Class Diagram

The following class diagram was designed to clearly illustrate the internal structure of the ADEXOS platform, which is based on the principles of object-oriented programming (OOP), as well as the correspondence between the data model derived from the ENT-REL diagram (entity-relationship diagram) and the code level. The attributes of the system's classes have been mapped one-to-one to the database schema, while their methods and the structural relationships between them have been modeled in accordance with UML standards.

To adhere to the principle of "separation of concerns" and fully meet the requirements of the section (inheritance, composition, aggregation), a total of eight base classes were designed:

- **Inheritance:** To avoid code duplication, an abstract user superclass was designed, which contains common user data (full_name, email) in the system. The Student and Advisor subclasses, which have specific roles and permissions, were derived from this main class.
- **Composition:** A strong ontological link was established between the "StudyPlan" class and the "Task" class. When a study plan is deleted from the system, the subtasks associated with that plan are also completely removed from memory.
- **Aggregation / Association:** The management of "Student" objects by the "Advisor" class, as well as the "Student" class's association with the "ErrorQuestion," "MockExam," and "Badge" gamification module objects, have been modeled as standard relationships (associations).

These 8 classes, which constitute the system's fundamental data structure, are composed of the entities User, Student, Advisor, StudyPlan, Task, ErrorQuestion, MockExam, and Badge, respectively. The detailed methods and properties of the classes are presented in the diagram in Figure 7.

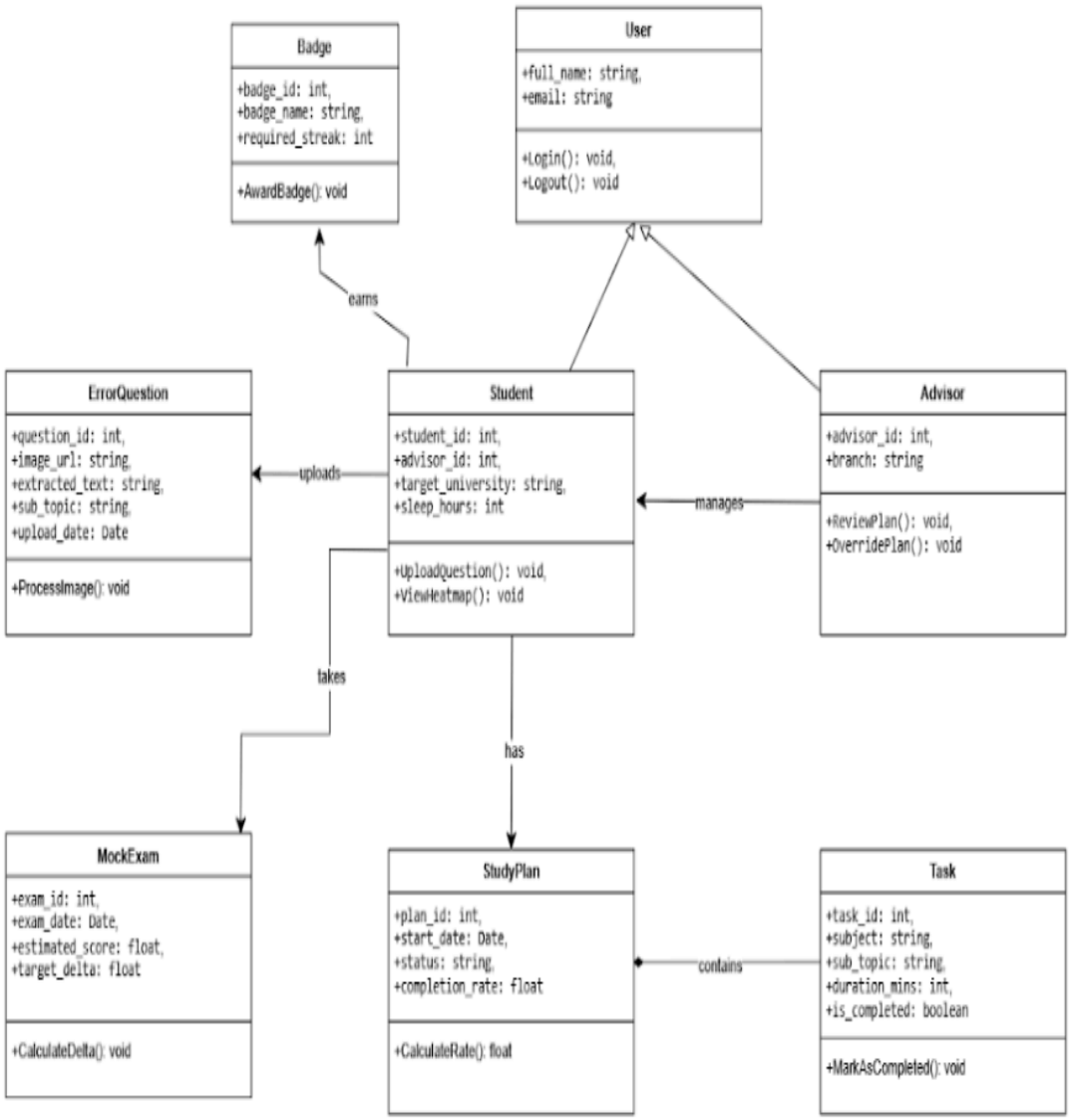



*Figure 7: ADEXOS Platform Class Diagram*

### 7.3 Sequence Diagrams

To illustrate the dynamic behavior of the ADEXOS system over time, as well as the order in which messages are transmitted (Message Passing) between layers (server, microservice, database) and external services, sequence diagrams have been developed for the two most critical use cases defined in Chapter 6.

**Sequence Diagram 1: Uploading and Classifying Incorrect Questions Using OCR (Ref.: UC1)**

This diagram models the asynchronous process from the download of a question that the student was unable to answer through to analysis by artificial intelligence and storage in

the ERROR_QUESTION table of the database. In accordance with the architectural security rule (NFR-02), it has been clearly specified that communication with the external API takes place exclusively via the isolated backend (C#).

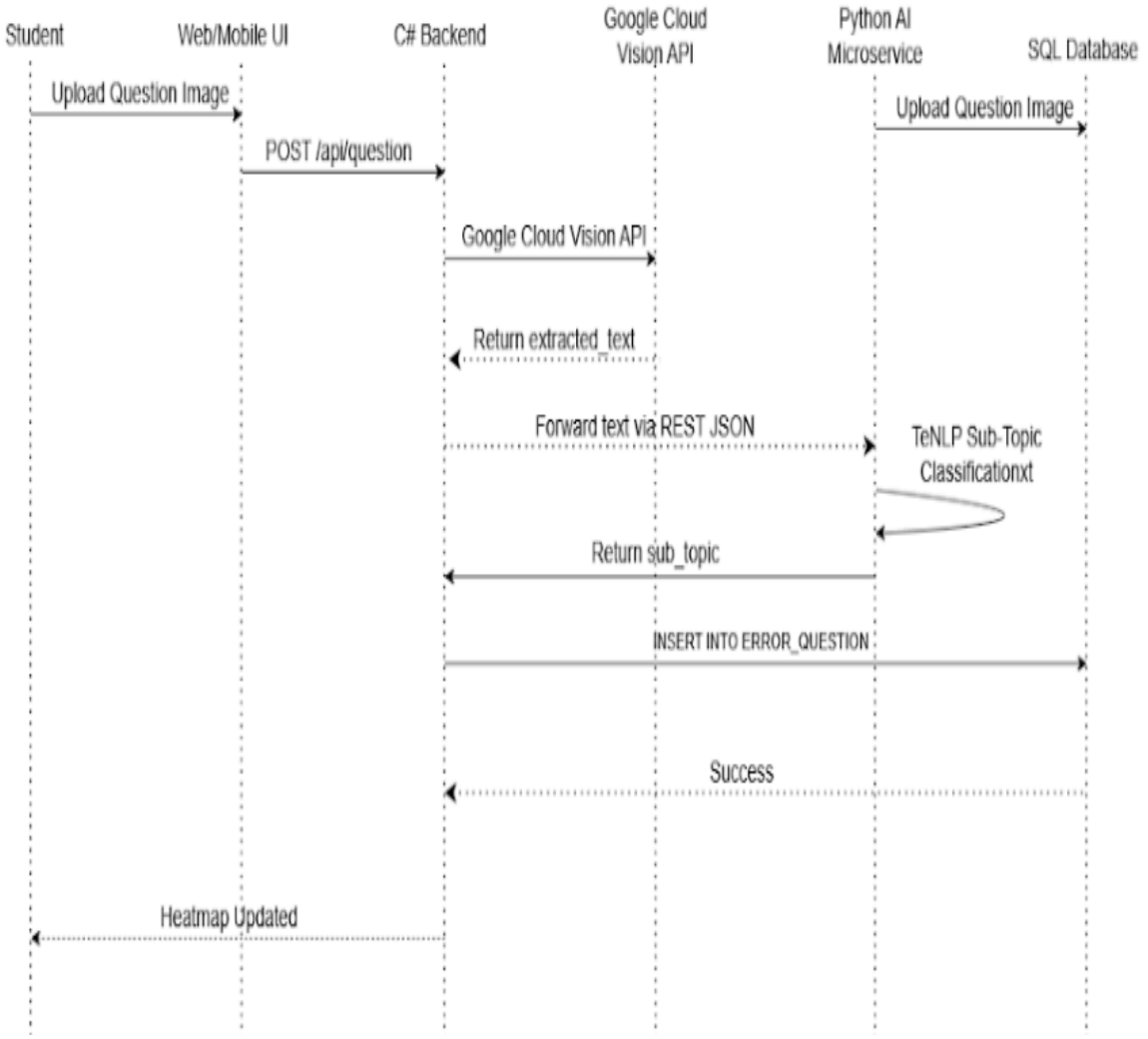



*Figure 8: Sequence Diagram - Upload Question and OCR/NLP Classification*

**Sequence Diagram 2: Dynamic Route Calculation (Ref.: UC3)**

This diagram illustrates the autonomous workflow in which the system activates itself every night (at 11:59 PM) to verify the student's daily task completion rate (70% rule) and, if necessary, to recalculate the route (rerouting).

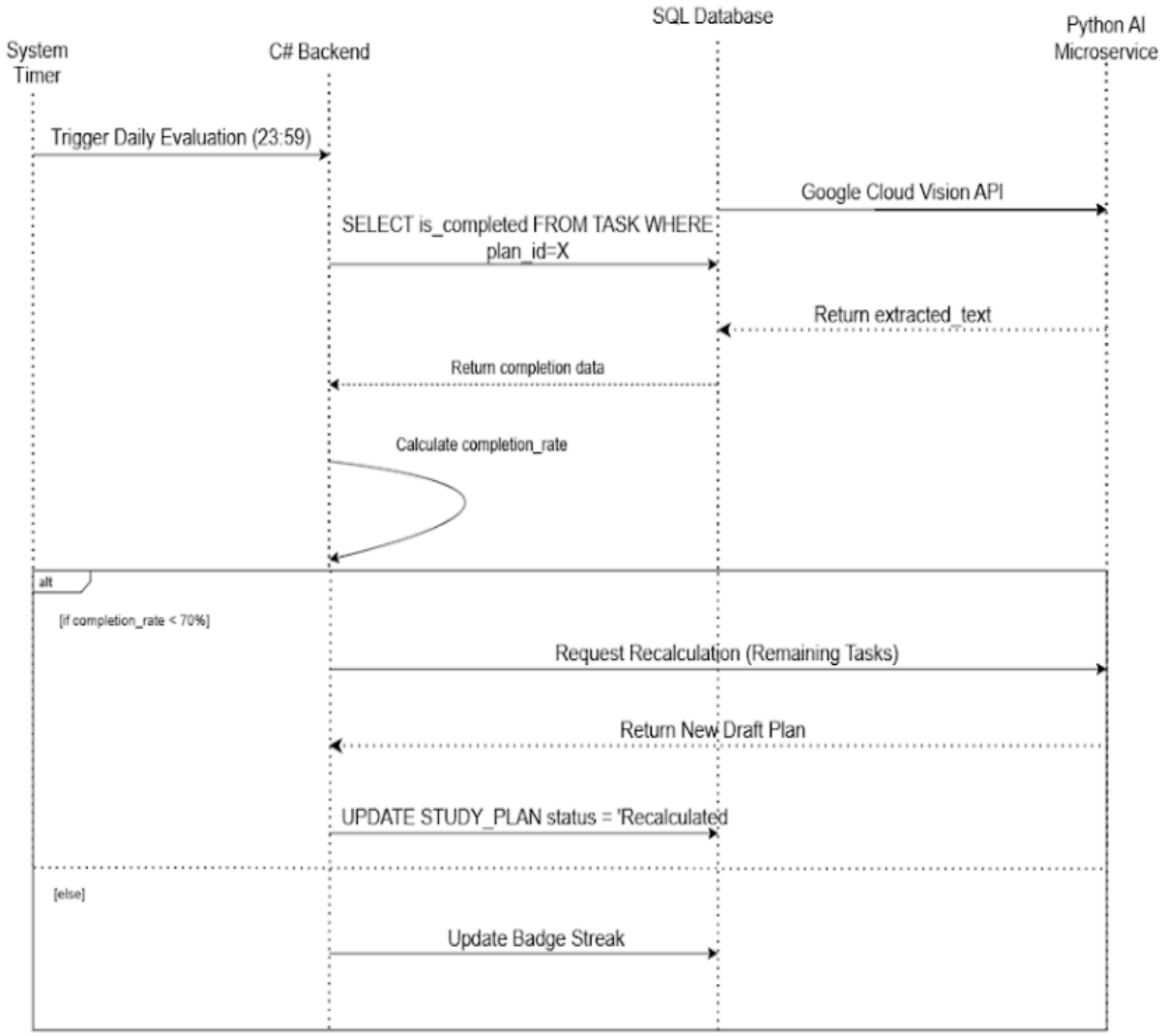



*Figure 9: Sequence Diagram - Daily Task Evaluation and Dynamic Route Recalculation*

### 7.4 Activity Diagram

ADEXOS's most critical educational workflow—namely, the "creation of a work plan (sprint) by artificial intelligence and the teacher validation process"—has been modeled using an activity diagram to make parallel processes and decision nodes (decision nodes).

This diagram illustrates how we integrate the "teacher" factor into the process as a control and verification mechanism (Human-in-the-Loop) to prevent the artificial intelligence from acting in a completely uncontrolled manner.

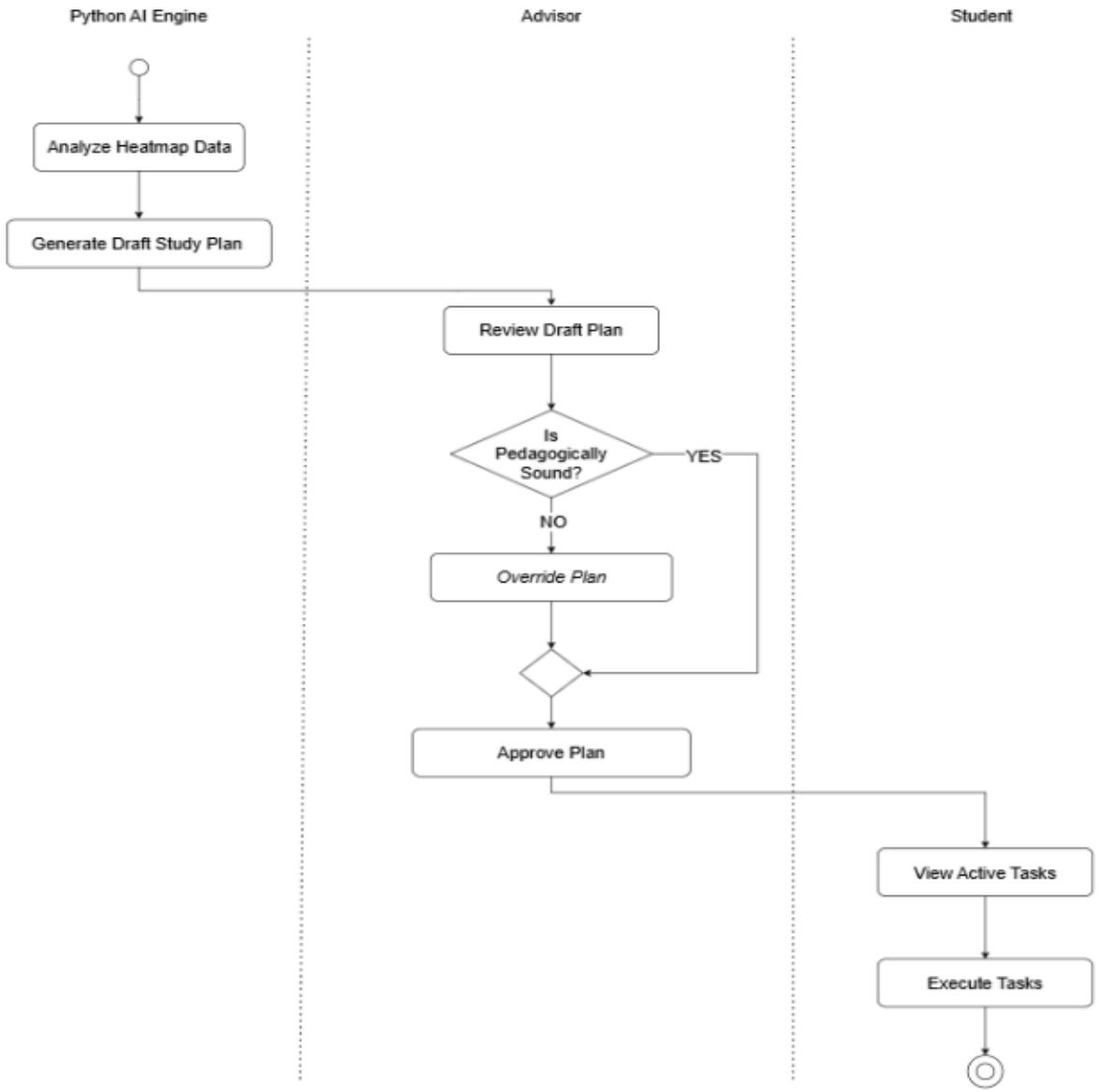



*Figure 10: Activity Diagram - Draft Plan Generation and Advisor Review Process*

### 7.5 State Machine Diagram

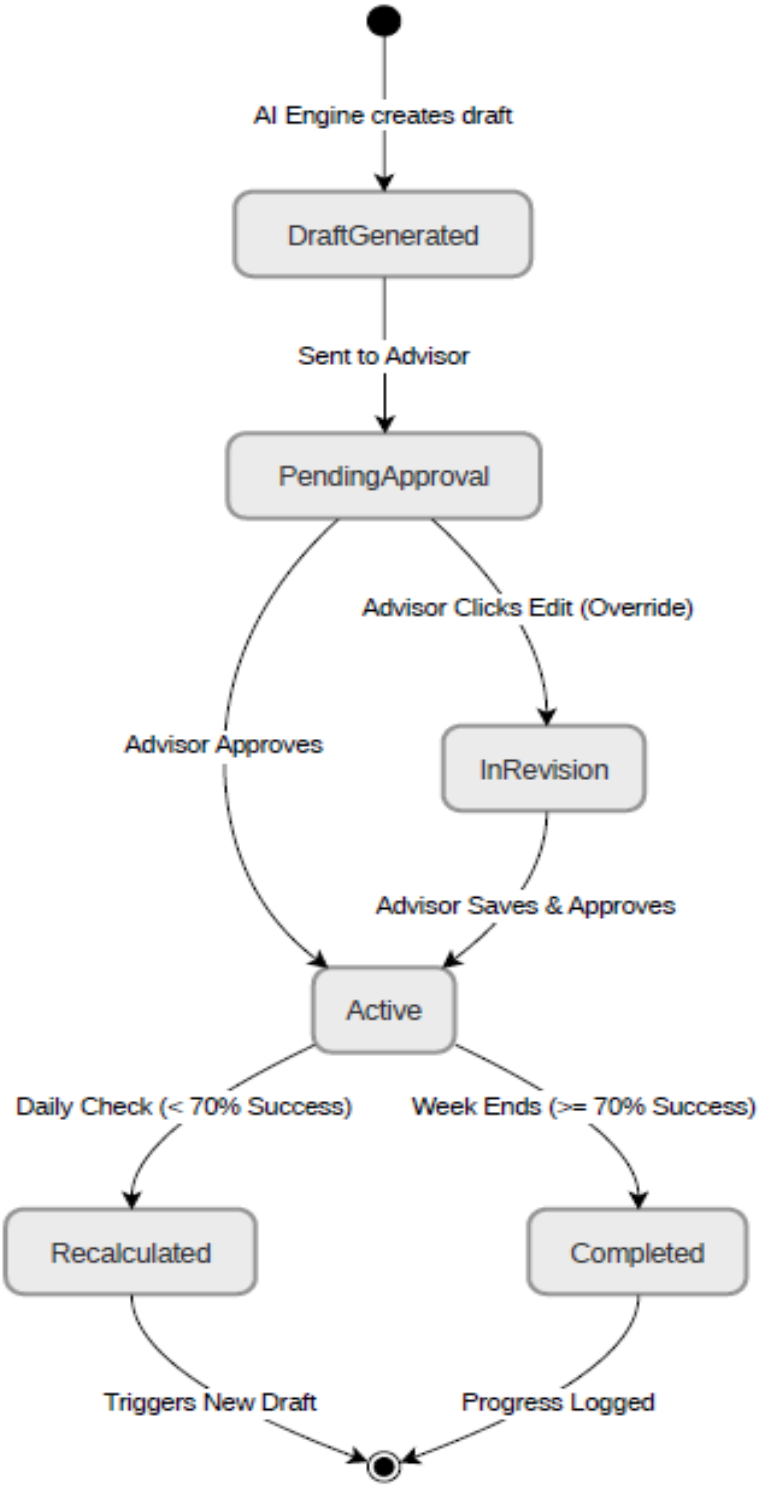



*Figure 11: State Machine Diagram - Study Plan Lifecycle*

The State Machine Diagram maps the lifecycle and stages of the Work Plan—the most dynamic object on the ADEXOS platform—within the system. A work plan passes through the following states within the system:

- **DraftGenerated:** This is the initial state in which the AI engine generates the weekly schedule for the first time by analyzing the student's heatmap data and constraints (sleep/school hours).
- **PendingApproval:** This is the stage where the generated draft is not sent directly to the student but is routed to the guidance counselor's dashboard to await pedagogical review.
- **InRevision (Override):** This is a temporary state where the guidance teacher makes manual adjustments to the schedule, such as changing class times or subjects. Once the teacher saves the changes, the plan immediately transitions to the "Active" status.
- **Active:** This is the status where the plan, approved by the guidance teacher, appears as daily tasks on the student's own dashboard and is ready for implementation.
- **Recalculated:** This status is triggered when the daily task completion rate drops below 70%. The current plan's execution is paused, and the system restarts the cycle by requesting the AI to generate a new draft for the following day.
- **Completed:** This is the final status where the student has successfully achieved 70% or more of their weekly goals, the process has concluded successfully, and the data has been transferred to the gamification module.

### 7.6 Component Diagram

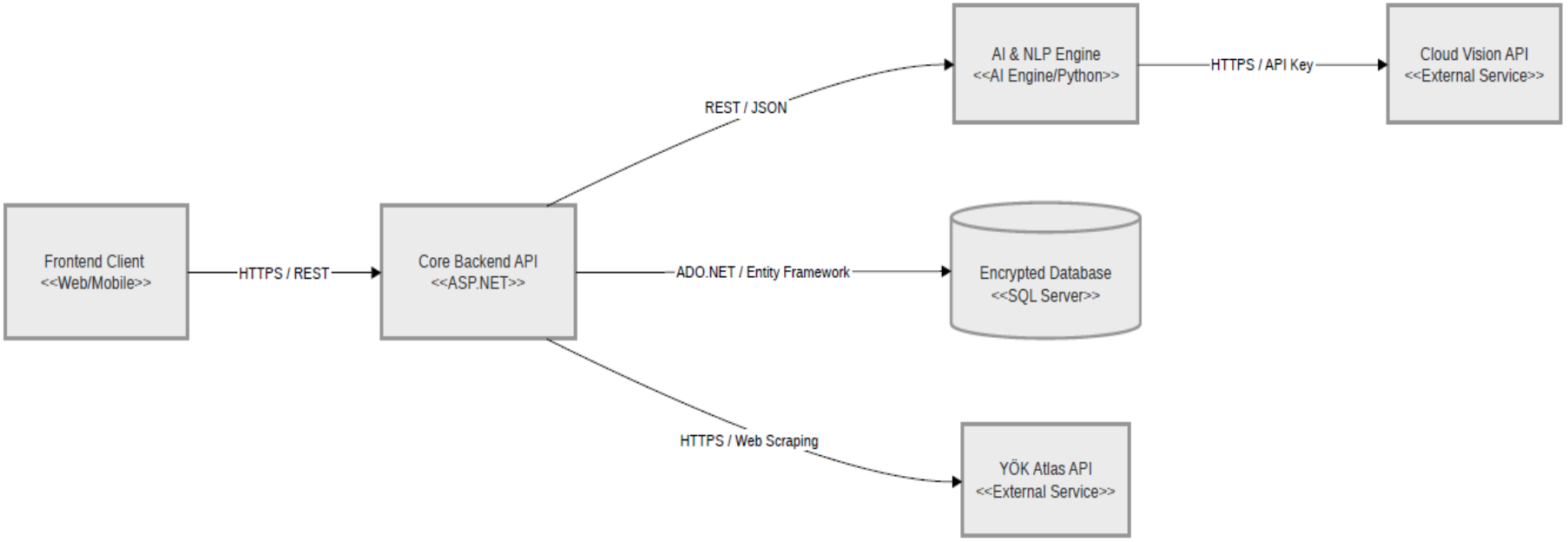



*Figure 12: ADEXOS Platform Component Diagram*

### 7.6.1 Component Descriptions

- **Frontend Client (Web/Mobile UI):** This is the layer through which students and guidance counselors interact with the system. To ensure security and integrity during data transmission, it communicates with the backend exclusively via secure HTTPS/REST protocols.

- **Core Backend API (C# ASP.NET):** It serves as the system's brain and primary orchestrator. It handles user authentication, business logic, and routing; manages database queries; and coordinates data with the AI module.

- **AI & NLP Engine (Python):** An independent module responsible for computationally intensive tasks such as generating practice exams, recalculating study routes, and analyzing OCR text. It communicates with the C# backend via JSON-based REST interfaces.

- **Encrypted Database (SQL Server):** This is the central repository where all data—including user profiles, heatmap records, and curriculum details—is stored. In accordance with KVKK standards, data is encrypted using AES-256.

- **External APIs:** The platform utilizes two critical external services: the Cloud Vision API, which extracts raw text from images, and the YÖK Atlas Data Service, which provides up-to-date university admission score thresholds.

## 7.7 Deployment Diagram

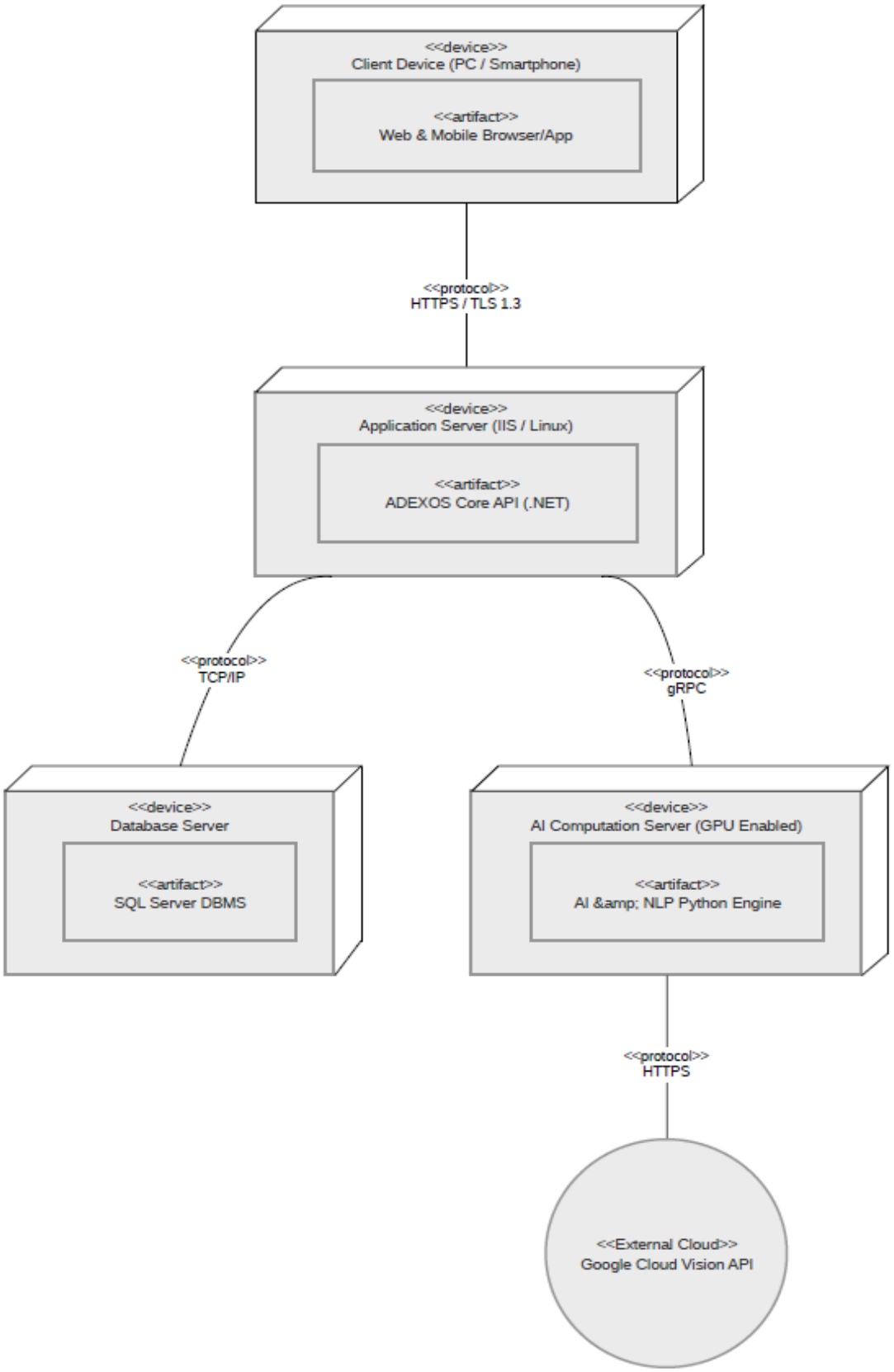



*Figure 13: ADEXOS System Deployment and Infrastructure Diagram*

### 7.7.1 Node and Network Descriptions

- **Client Device (Node):** These are the personal computers or smartphones that students and mentor teachers use to access the platform. The software artifact running on them is the ADEXOS interface.
- **Application Server (Node):** This is the main server hosting the C# ASP.NET core API, which forms the backbone of the system. It handles requests from clients and communicates with the outside world over a secure internet connection (HTTPS/TLS 1.3).
- **AI Computation Server (Node):** This is hardware—preferably GPU-enabled—where resource-intensive machine learning algorithms, such as route recalculation and mock exam generation, are executed. It communicates with the main server via the gRPC protocol over a high-speed internal network (Internal LAN).
- **Database Server (Database Server - Node):** Physical or virtual hardware running the SQL Server management system. It communicates securely with the main application server over TCP/IP Port 1433 within the closed internal network; direct external access is blocked to ensure the highest level of security.
- **External Cloud Service (Google Cloud Vision API):** This is an independent cloud infrastructure managed by Google where the platform performs OCR operations externally without overloading its own main servers. The ADEXOS main server sends the raw image to this service directly over the internet via a secure HTTPS protocol and receives the processed text data in return.

## 7.8 Entity-Relationship(ER) Diagram

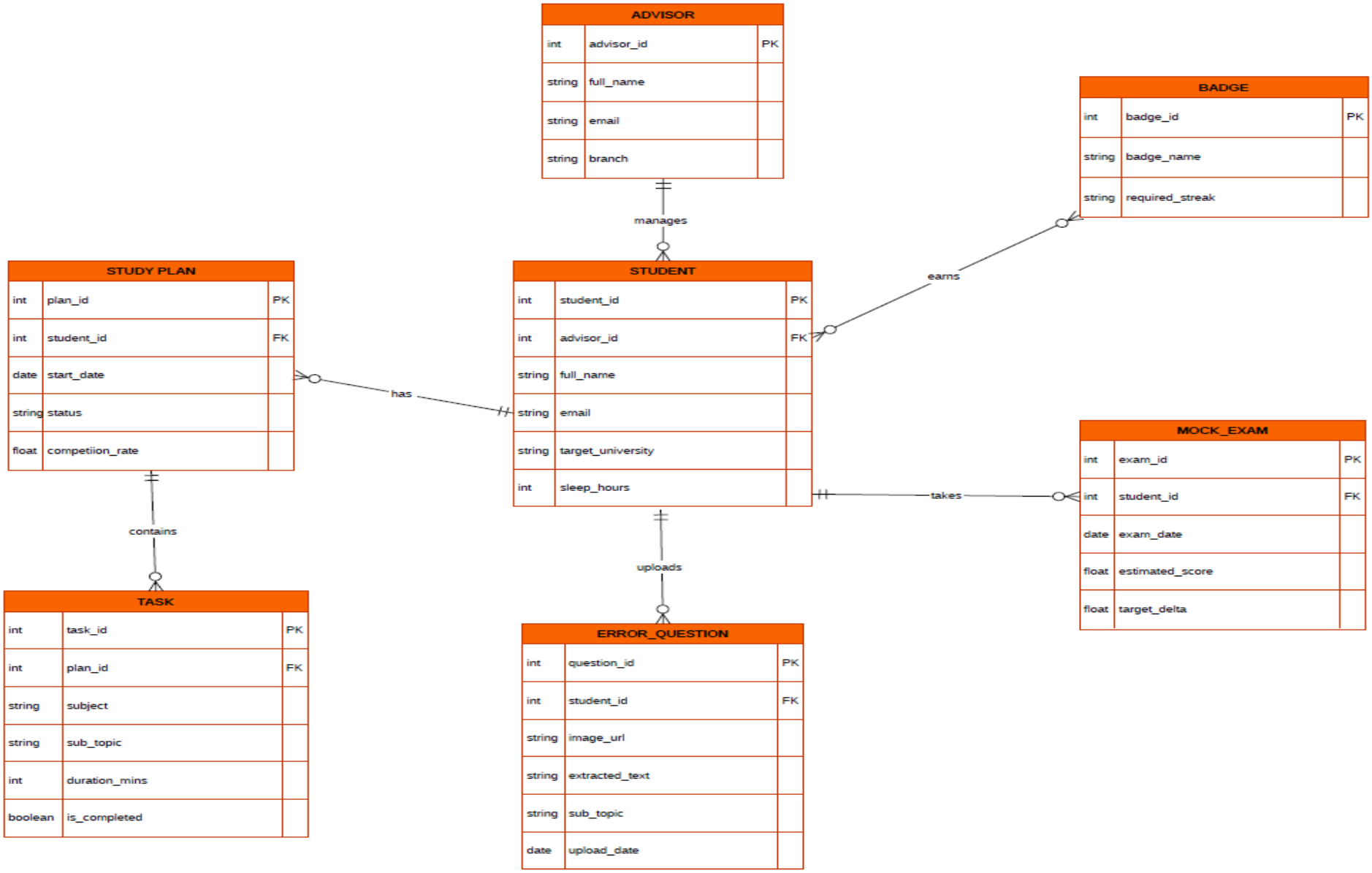

ADVISOR
int advisor_id PK
string full_name
string email
string branch
manages
BADGE
int badge_id PK
string badge_name
string required_streak
earns
STUDY PLAN
int plan_id PK
int student_id FK
date start_date
string status
float competiion_rate
has
STUDENT
int student_id PK
int advisor_id FK
string full_name
string email
string target_university
int sleep_hours
takes
MOCK_EXAM
int exam_id PK
int student_id FK
date exam_date
float estimated_score
float target_delta
contains
uploads
TASK
int task_id PK
int plan_id FK
string subject
string sub_topic
int duration_mins
boolean is_completed
ERROR_QUESTION
int question_id PK
int student_id FK
string image_url
string extracted_text
string sub_topic
date upload_date

*Figure 14: ADEXOS Database Entity-Relationship (ER) Diagram*

The Entity-Relationship (ER) Diagram represents the logical data model and relational database schema (RDBMS) of the ADEXOS platform. Data integrity between tables is ensured through Primary Keys (PK) and Foreign Keys (FK).

- **One-to-Many Relationships:** In this hierarchical structure, which forms the core of the system, each advisor (ADVISOR) can manage multiple students (STUDENT). Similarly, a student may have multiple study plans (STUDY_PLAN), error question records (ERROR_QUESTION), and mock exams (MOCK_EXAM) over time.
- **Many-to-Many Relationships:** Defines the relationship between students and Badges (BADGE), the system's gamification module.
- **Data Scope:** The schema is designed to include all necessary sub-topic, duration, and completion (is_completed) metrics so that the AI engine can power the "Recalculate Route" and "Heatmap" features.

### 7.8.1 Database Table and Attribute Descriptions

The function of each table in the ADEXOS database schema and the technical details of the attributes they contain are described below:

**1. STUDENT Table**

It stores the personal data of students using the platform, their academic goals, and the time constraints required for the AI algorithm.

- **student_id (PK):** It is the primary key that uniquely identifies each student in the system.
- **advisor_id (FK):** Indicates the student's assigned advisor; links to the ADVISOR table (1:N relationship).
- **full_name:** The student's first and last name.
- **email:** This is the unique communication address used for system login and notifications
- **target_university:** Information about the university and department the student is aiming for (used in YÖK Atlas Delta simulation calculations).

- **sleep_hours:** The student's daily sleep duration limit (the chronobiological data used by the AI when creating weekly schedules).

**2. ADVISOR Table**

Stores the account information and areas of expertise of the guidance counselors in the system.

- **advisor_id (PK)**: The primary key that uniquely identifies the guidance counselor.
- **full_name**: The advisor's first and last name.
- **email**: The email address used for logging into the teacher dashboard and for internal system communication.
- **branch**: The advisor's area of expertise or subject (e.g., Counseling, Mathematics).

**3. STUDY_PLAN Table**

Records the status of weekly study plans generated by artificial intelligence or revised by a mentor teacher.

- **plan_id (PK):** The unique identifier for each weekly study plan.
- **student_id (FK):** The foreign key indicating which student the plan belongs to; it references the STUDENT table.
- **start_date:** The start date of the relevant study week.
- **status**: Holds the plan's current status (takes the values "Draft", 'Active', "Recalculated").
- **completion_rate**: The overall completion percentage of the plan at the end of the week (in float format).

**4. TASK Table**

Weekly study plans contain subtasks such as daily study sessions, watching videos, or taking quizzes.

- **task_id (PK):** The unique identifier for each subtask.
- **plan_id (FK)**: The foreign key that links the task to the weekly plan it belongs to; it references the STUDY_PLAN table.

- **subject:** The main subject category (e.g., Mathematics, Physics, Biology).
- **sub_topic:** The lowest-level topic heading within the subject (e.g., Product Rule in Calculus). This is a critical attribute for the proper feeding of the heatmap.
- **duration_mins:** The estimated time required to complete the task (in minutes).
- **is_completed:** A logical (boolean) value indicating whether the task has been completed by the student.

**5. ERROR_QUESTION Table**

It stores images of incorrect questions uploaded by students and the AI analysis results derived from these images.

- **question_id (PK):** The unique identifier for each incorrect question uploaded to the system.
- **student_id (FK):** The foreign key indicating which student uploaded the question; it references the STUDENT table.
- **image_url:** The secure file path of the question image on the cloud storage server.
- **extracted_text:** The raw text and formula data extracted from the image by the OCR module.
- **sub_topic:** The MEB/ÖSYM subtopic label assigned to the question by the NLP module after analyzing the raw text.
- **upload_date:** The date the question was saved to the system..

**6. MOCK_EXAM Table**

Stores practice exams generated by artificial intelligence based on past ÖSYM frequency data, along with students' performance metrics on these exams.

- **exam_id (PRIMARY KEY):** The unique identifier of the generated or completed practice exam.
- **student_id (FOREIGN KEY):** The student ID of the student who took the exam; linked to the STUDENT table.
- **exam_date:** The date the practice exam was completed.
- **estimated_score:** The estimated YKS score calculated by the system based on the exam’s raw scores.

- **target_delta:** The mathematical difference between the student's target score and the estimated score obtained from the exam (feeds the target simulation graph).

**7. BADGE Table**

Contains definitions for the rule-based reward mechanism designed to motivate students.

- **badge_id (PK):** The badge's unique identifier.
- **badge_name:** The badge's name in the system (e.g., "Discipline Ambassador").
- **required_streak:** The rule threshold specifying the number of consecutive days a student must complete their daily tasks 100% to earn the badge (e.g., 7 days). The M:N relationship between students and badges is resolved in the code layer via a join table, as required by relational database rules.

## SECTION 8: IMPLEMENTATION PLAN

This section provides a detailed description of the transition strategy from the design phase of the ADEXOS platform to the programming and deployment phase. The chosen technology stack, programming standards, and module integration processes have been justified in accordance with software development principles.

### 8.1 Technology Stack

Since the system meets various requirements (web presentation, relational data management, AI calculations), the "right tool for the job" principle was applied, and the following technology stack was selected:

#### 8.1.1 Backend(Business Layer) : C# & ASP.NET Core

- Justification: Due to its powerful, object-oriented structure, it was chosen to handle the system's central orchestration, manage user permissions, and execute enterprise-wide operations such as "task synchronization." For database operations, **Entity Framework Core**, the powerful ORM tool within the C# ecosystem, is used.

#### 8.1.2 AI Microservice (AI Layer): Python, FastAPI, TensorFlow & LangChain

- Justification: Python is the industry standard in the fields of machine learning and natural language processing (NLP). TensorFlow and Scikit-Learn algorithms are used to process student heatmap data and perform dynamic route calculations (rerouting. To break down the question texts into subtopics, LangChain is integrated into the NLP pipeline, and this service is easily connected to the main server via FastAPI.

#### 8.1.3 Database : MS SQL Server

- Justification: To ensure data integrity (ACID principles) and the relational structure of critical data such as student assignments, test results, and teacher validations, a robust RDBMS (relational database management system) was selected.

#### 8.1.4 Frontend (presentation layer): React.js

- Justification: React.js was chosen to quickly render the system's highly interactive user interfaces (UI), such as the heatmap, target simulation charts, and daily assignment dashboard, using a component-based architecture.

#### 8.1.5 External services (APIs): Google Cloud Vision API & YÖK Atlas

- Justification: To avoid wasting time training a model from scratch for the OCR (optical character recognition) process (separation of concerns), Google Vision was chosen; to ensure the official nature of the minimum university grades used in target simulations, data from YÖK Atlas was integrated into the system

### 8.2 Coding Standards

To ensure code consistency across the 2-person team during the development process and to minimize technical debt, strict standards were established.

### 8.2.1 Naming Conventions

- **C# / Backend:** PascalCase is used for class and method names (e.g., CalculateDelta(), StudyPlan), while camelCase is used for local variables and parameters (e.g., targetScore).
- **Python and SQL Database:** In accordance with the ER diagram in Chapter 7, the use of snake_case is mandatory for database tables and variables in the Python microservice (e.g., error_question, completion_rate, sleep_hours).
- **API Communication:** For REST API data packets, which use two different languages, JSON has been defined as the standard format.
- **Code Documentation**: To transparently document the C#-side REST API endpoints, the Swagger (OpenAPI) interface is integrated into the system. On the Python side, "docstrings" are used to explain complex AI methods.

### 8.2.2 Version Control Strategy

To avoid the R10 risk (merge conflict) defined in Section 5, a standard professional Git workflow is used in the project:

- **Branching:** The main branch always contains production-ready code. Development is carried out via the "develop" branch, and for each new feature (e.g., OCR integration), separate branches such as "feature/ocr-integration" are created.
- **Rule regarding pull requests (PRs):** Before being merged into the main branch, the code must be reviewed by at least one team member.

## 8.3 Module Decomposition

The architecture designed in Chapter 7 and the use case scenarios were decomposed, during the coding phase, into the following concrete implementation modules:

- **Auth & User Management Module (C#):** Manages user logins, authorization for mentor/student roles, and encryption operations (AES-256).
- **Task and Routing Orchestrator (C# & SQL):** This is the main orchestration module that calculates the student's daily task completion rate (70% rule), activates at 11:59 PM via a Cron job (scheduled task), queries the database, and, if necessary, sends the "Recalculate" command to the Python service.

- **NLP & OCR Engine Module (Python):** A machine learning module that, when the "UploadQuestion" action is triggered, transmits the image sent via C# to the Google Vision API, analyzes the returned text using LangChain, and identifies the subtopic (e.g., "Derivation").
- **Gamification and Dashboard Module (React & C#):** This module enforces badge rules in the background and displays streak counts in an animated manner within the user interface (UI).

### 8.4 Build and Deployment Pipeline (CI/CD Approach)

To improve the system's testability and sustainability, a continuous integration and continuous deployment (CI/CD) pipeline is planned.

- **Continuous Integration (CI):**A CI pipeline is set up using GitHub Actions. Every time developers "push" their code to the "develop" branch, unit tests are automatically run in the background to verify that the application can be built. This prevents faulty code from affecting the main project.
- **Continuous Deployment (CD):** The client layer, the C# backend, and the Python AI service are converted into independent **Docker** containers. This containerization eliminates the "It worked on my machine" problem, and the system is designed to be deployed seamlessly to cloud servers, in line with the 99.9% availability (NFR-04) promised in Chapter 6.

## SECTION 9: TESTING PLAN

### 9.1 Test Strategy

To ensure the project's quality assurance, a testing process was implemented at four different levels, using the V-Model testing approach as a reference. This approach made it possible to verify whether the system met both technical requirements and user expectations.

**• Unit Testing:**

At this stage, the software's smallest components—methods and functions—were isolated and tested. The accuracy of the underlying computational algorithms was verified using the xUnit/NUnit framework for C# and the PyTest framework for Python. During this

process, it was confirmed that critical mathematical operations—primarily delta calculations and success rates—produced the expected outputs.

• **Integration Testing:**

Integration tests examined data exchange and communication processes between different software modules. In particular, API communications between the C#-based backend, Python AI services, and the SQL database were included within the scope of the tests. During this process, data consistency and communication reliability between modules were verified..

• **System Testing:**

As part of the system testing, the ADEXOS platform was evaluated to determine whether it functions correctly as a single, end-to-end system. Large-scale workflows, such as dynamic route recalculation (rerouting), exam simulation creation, and the synchronized execution of user operations, were tested at this level.

• **Acceptance Testing:**

Acceptance testing was conducted to determine whether the system meets end-user requirements. The tests, conducted from the perspectives of both students and guidance counselors, focused on the user interface, usability, system responsiveness, and overall user experience. As a result of this phase, it was confirmed that the system aligns with user expectations.

### 9.2 Test Cases

The 20 test scenarios developed and executed in accordance with the 4-phase test strategy outlined in Section 9.1 are listed below.

### 9.2.1 Unit Testing

| ID | Related FR | Description | Preconditions | Test Steps | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|---|---|
| **TC-01** | FR-01 | Unit test for the OCR module's raw text extraction function. | A mock question image has been uploaded to the test environment. | 1. Upload the image to the OCR function.<br>2. Check the output. | The text in the image must be returned as a complete string. | Verified using PyTest: It was confirmed that the function returns the image as a string without any loss of quality. | **PASS** |
| **TC-02** | FR-02 | Test for correctly assigning subtopic tags to an NLP model. | A mock string containing the words “derivative product” is provided. | 1. Send the string to the NLP tagging function. | The function should return the label “Product Rule for Derivatives.” | Verified with PyTest: The algorithm assigned the correct label with a confidence score of 98%. | **PASS** |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **TC-03** | FR-04 | Unit test of the Target-Delta mathematical difference calculation method. | The student's estimated score is 350, while the YÖK Atlas cutoff score has been entered into the system as 400. | 1. Call the CalculateDelta(350, 400) method. | The method must return exactly 50.0 as a float value. | Tested with xUnit: It was verified that the method correctly returns the expected delta value of 50.0. | **PASS** |
| **TC-04** | FR-08 | 70% task completion threshold calculation test. | It is recorded that 6 out of the student's 10 assignments have been completed. | 1. Run the CalculateDailyRate() function. | The method should return a value of 60%. | Tested with xUnit: The return rate was calculated at 60%, triggering the recalculation threshold. | **PASS** |
| **TC-05** | FR-10 | Test of the function to increment the gamification (Streak) counter. | The student's current streak is defined as 6. | 1. Complete the daily tasks.<br>2. Call IncrementStreak(). | The value of the `streak` variable must be 7. | Tested with xUnit: The counter value was updated to 7 in memory. | **PASS** |

*Table 6: Unit Testing Cases*

### 9.2.2 Integration Testing

| ID | Related FR | Description | Preconditions | Test Steps | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|---|---|
| **TC-06** | FR-01 | Connection between the Python AI engine and the external OCR API (Cloud Vision). | A valid API key is configured in the Python environment. | 1. Send the image file to the external API using an HTTP POST request. | The API must pass authentication and return JSON. | The Python microservice retrieved text data from an external API with a 200 OK response. | **PASS** |
| **TC-07** | FR-07 | React Frontend and C# API task synchronization integration. | There are approved tasks in the database. | 1. Send a GET request to /api/tasks/approved from the UI. | The C# API must return data in valid JSON format. | The data exchange (fetch) between the client and the C# backend was successful. | **PASS** |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **TC-08** | FR-08 | Triggering the Python Rerouting service of the C# API using gRPC. | The student's task completion rate is 60%. | 1. At 11:59 PM, C# sends a gRPC signal to the Python microservice. | The Python service should start calculating the route. | A signal was successfully received via the inter-module gRPC tunnel, and the algorithm was triggered. | **PASS** |
| **TC-09** | FR-04 | HTTP integration of the C# API with the YÖK Atlas external data API. | The university ID has been set to 1040 (Boğaziçi). | 1. Send a data retrieval request to the YÖK Atlas API. | The service should provide the latest cutoff score data. | The data from the target university was successfully retrieved as JSON via the C# API. | **PASS** |
| **TC-10** | NFR-02 | Testing the integration of AES-256-encrypted data writing between C# and SQL Server.<br>. | The student's exam score has been entered into the system. | 1. Write to the database using Entity Framework.<br>2. Query raw data from SQL. | The raw data in the database must be in encrypted (hashed) format. | C# and SQL communication has been verified; the data is encrypted (ciphertext) | **PASS** |

*Table 7: Integration Testing Cases*

### 9.2.3 System Testing

| ID | Related FR | Description | Preconditions | Test Steps | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|---|---|
| **TC-11** | FR-03 | Collect end-to-end data and generate a heatmap. | The database contains a large amount of erroneous question data (Derivative) belonging to the student. | 1. Run the analysis module.<br>2. Start the heat map rendering process. | The corresponding checkbox should be set to “Red” based on the data. | E2E test passed: The database data was processed, and the system rendered the relevant block in red on the UI. | **PASS** |
| **TC-12** | FR-05 | System-wide generation of AI draft schedules aligned with sleep schedules. | The student's sleep restriction is set for 11:00 PM to 7:00 AM. | 1. Trigger the AI Draft Plan generation (Sprint). | The task blocks created should not fall within this time frame. | E2E test successful: The system read the sleep timestamps; there were no conflicts. | **PASS** |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **TC-13** | FR-06 | The system should execute the Teacher Override (manual revision) workflow. | The draft plan is on the teacher's editing screen. | 1. Change the duration from 60 minutes to 90 minutes.<br>2. Click “Save.” | The system should synchronize the changes without causing any errors. | End-to-end test successful: Transaction completed; data has been updated to 90 minutes across all systems. | **PASS** |
| **TC-14** | FR-08 | The system performs end-to-end dynamic route recalculation. | The student did not meet the daily task goal (60%). | 1. Set the system time to 11:59 PM.<br>2. Monitor the process. | The system should cancel the old plan and generate a new route based on the remaining week. | E2E test successful: Old tasks were canceled in the system clock, and a new navigation route was created. | **PASS** |
| **TC-15** | FR-09 | The system's algorithmic mock exam compilation. | According to the student's profile, the student is particularly weak in “Physics.” | 1. Run the “Generate Practice Test” main function. | The exam should consist of 40 questions, and there should be more questions on weaker topics. | E2E test successful: A 40-question exam session, organized by difficulty level, was launched without errors. | **PASS** |

*Table 8: System Testing Cases*

### 9.2.4 Acceptance Testing

| ID | Related FR | Description | Preconditions | Test Steps | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|---|---|
| **TC-16** | FR-06 | Availability of the guidance counselor's approval on the UI. | The plan is in the “Pending Approval” status. | 1. The teacher clicks the “Confirm” button. | The plan should be approved immediately and display a success message on the interface. | The user experience (UX) was seamless, and the confirmation process was reflected in the interface without any delay. | **PASS** |
| **TC-17** | FR-10 | A demonstration of the gamification badge animation. | The student's streak has reached 7 days. | 1. The student logs in to the dashboard. | An interface animation indicating that you have earned a badge should appear on the screen. | The necessary UI requirements have been met, and the badge animation plays smoothly from the student's perspective. | **PASS** |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| **TC-18** | FR-11 | Activating a user account in the Admin panel. | A new teacher has submitted a registration request to the system. | 1. Log in to the admin panel.<br><br>2. Set the account to “Active” in the UI. | The relevant teacher must have been granted access to the system. | The admin interface ran smoothly, the account status was updated, and access was granted. | **PASS** |
| **TC-19** | FR-01 | The student uploads a photo using the camera via the mobile/web interface. | The student has granted the device permission to access the camera. | 1. Take a photo of the question.<br><br>2. Click the “Upload” button. | A “Photo successfully captured” confirmation dialog (toast) should appear on the interface. | The image captured by the mobile device's camera was successfully transmitted to the system without freezing the user interface.. | **PASS** |
| **TC-20** | FR-04 | Readability of the Target-Delta simulation graph interface. | The simulation process has been completed in the background. | 1. Go to the “Performance and Simulations” tab. | The distance to the target should be displayed to the user using a clear, visual graph. | The charts were rendered in a user-friendly format (red/green bars), and the screen's readability was verified. | **PASS** |

*Table 9: Acceptance Testing Cases*

### 9.3 RTM Verification

The Requirements Traceability Matrix (RTM) below demonstrates that each Functional Requirement (FR) defined in Section 6 corresponds to at least one—and in most cases, more than one—test scenario within the framework of software engineering standards. This structure has proven that test coverage has reached 100% (Full Test Coverage).

| Req ID | Requirement Name & Description | Type | Priority | Test Case ID (TC) Match |
|---|---|---|---|---|
| **FR-01** | OCR Image Processing | Functional | High | TC-01, TC-06, TC-19 |
| **FR-02** | NLP Sub-Topic Classification | Functional | High | TC-02 |
| **FR-03** | Visual Heatmap Generation | Functional | Medium | TC-11 |
| **FR-04** | Target-Delta Simulation | Functional | Medium | TC-03, TC-09, TC-20 |
| **FR-05** | AI Dynamic Sprint Planning | Functional | High | TC-12 |
| **FR-06** | Advisor Review & Override | Functional | High | TC-13, TC-16 |
| **FR-07** | Task Board Synchronization | Functional | High | TC-07 |

| | | | | |
|---|---|---|---|---|
| **FR-08** | Smart Route Recalculation | Functional | High | TC-04, TC-08, TC-14 |
| **FR-09** | OSYM Historical Mock Exam | Functional | Medium | TC-15 |
| **FR-10** | Gamification Module | Functional | Low | TC-05, TC-17 |
| **FR-11** | Curriculum & Admin Management | Functional | Medium | TC-18 |
| **NFR-02** | Data Encryption | Non-Functional | High | TC-10 |

*Table 10: RTM Verification - Test Coverage Matrix*

## SECTION 10 : MAINTANENCE PLAN

This section outlines the long-term maintenance strategies to be followed to ensure the system's sustainability, performance, and up-to-date status after the ADEXOS platform is deployed to the production environment.

### 10.1 Anticipated Maintenance Types

In accordance with software engineering principles, four basic types of maintenance will be implemented throughout the ADEXOS system's lifecycle:

- **Corrective Maintenance:** This refers to the process of resolving errors that arise during the system's live operation. For example, this includes fixing instances where the OCR module misreads certain mathematical symbols in blurry photos, or applying emergency patches when a button on the student interface fails to work on mobile devices.

- **Adaptive Maintenance:** This is the type of maintenance the project will require most frequently. It is performed to ensure the system adapts to environmental changes in the external world or in the infrastructure. Examples include updating the system's database schema to align with official changes in the MEB curriculum or the ÖSYM's YKS format (such as changes in the number of questions or subject weightings), and ensuring that browsers (Chrome/Safari) comply with new security policies.

- **Perfective Maintenance:** Even if there are no errors in the system, this involves improving existing features or adding new functions based on feedback from students and guidance counselors. Examples of such improvements include optimizing the real-time route calculation time using artificial intelligence, refining the color scale of the heat map, or adding new gamification badges.

- **Preventive Maintenance:** This involves proactively reviewing the code and infrastructure to prevent potential system crashes or performance issues in the future. Given the anticipated increase in user traffic during the spring months leading up to the YKS exam, implementing query indexing in SQL Server and addressing accumulated technical debt through code refactoring can be considered within this scope.

### 10.2 Change Management Process

To prevent uncontrolled changes to the system (scope creep) and to ensure data integrity in the production environment, a formal Change Management Process will be implemented. Every change request goes through the following steps:

1. **Request:** Students or guidance counselors can submit an error report or a feature request by opening a support ticket or reviewing the system monitoring logs.

2. **Impact Analysis:** The development team analyzes the impact of the change in question on the C# backend or Python AI models, as well as the estimated effort and potential risks.

3. **CCB - Change Control Board:** A committee composed of project managers and technical leads reviews the impact analysis; they decide whether the change aligns with the system's objectives and then approve, reject, or defer the request to the next Sprint (Backlog).

4. **Implementation & Testing:** The approved changes are implemented in the develop branch and undergo the automated unit and integration tests defined in Section 9.

5. **Release & Deployment:** Code that passes the tests is merged into the main branch via the CI/CD pipeline and automatically deployed to the production environment with a new version number.

### 10.3 Version Numbering Scheme

The Semantic Versioning (SemVer) standard, which is frequently used in software development processes on the ADEXOS platform, has been adopted. This structure is based on the MAJOR.MINOR.PATCH format, and the system's first stable version has been designated as 1.0.0.

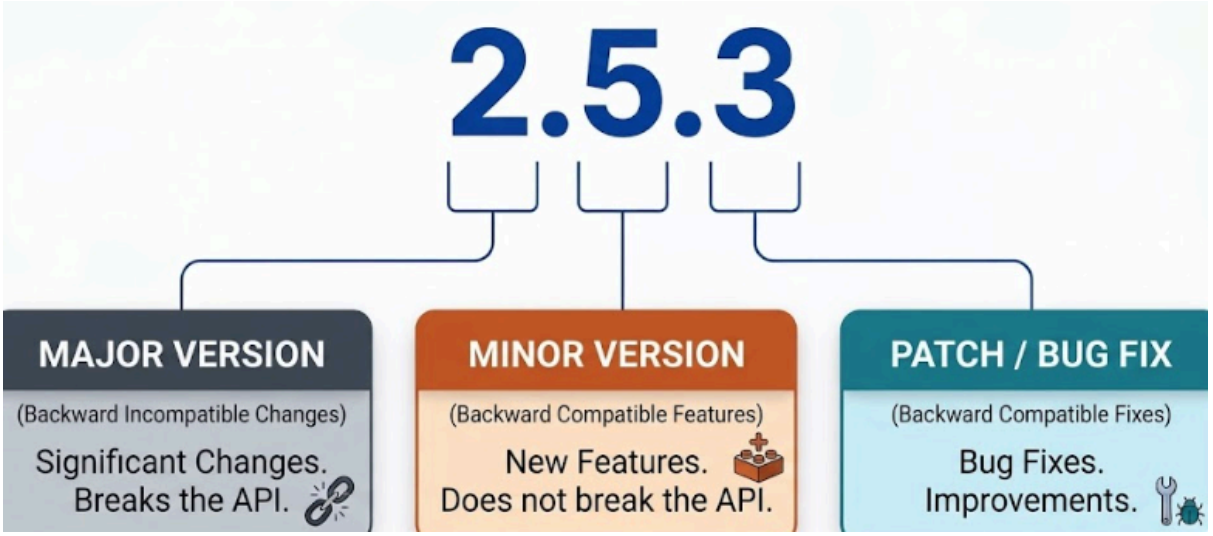



*Figure 15: Semantic Versioning (SemVer) Structure*

- **MAJOR VERSION (X.0.0):** It increases when fundamental changes are made at the architectural level that break backward compatibility in the system. Such updates require a significant redesign of the existing system architecture.

  Example: 1.4.2 → 2.0.0 (A complete transition to a new-generation AI model capable of analyzing open-ended questions.)

- **MINOR VERSION (0.X.0):** It is enhanced when new features or modules are added to the system while maintaining backward compatibility. It is used to improve functionality without disrupting the existing user experience.

  Example: 1.0.0 → 1.1.0 (Adding a bulk graphical reporting screen to the teacher dashboard or a new gamification badge.)

- **PATCH VERSION (0.0.X):** It is incremented when minor bug fixes, security patches, or interface improvements are made to the current system features. These updates do not fundamentally change system behavior.

  Example: 1.1.0 → 1.1.1 (Resolution of an algorithmic issue that caused the OCR module to incorrectly recognize the square root symbol.)

Thanks to this versioning approach, not only the software development process but also its long-term sustainability and its ability to adapt to changes in the ÖSYM and MEB systems have been documented.

## SECTION 11: ETHICAL, LEGAL, AND SUSTAINABILITY CONSIDERATIONS

This section evaluates, beyond the technical capabilities of the ADEXOS platform, aspects such as legal data protection, the ethical use of artificial intelligence and the contribution to global sustainable development goals.

### 11.1 Data Privacy and GDPR/KVKK Compliance

The ADEXOS system processes personal data, sleep patterns, exam results, and handwritten documents from its target audience, which consists of high school students. For this reason, the system strictly adheres to the standards of the European Union's General Data Protection Regulation (GDPR) and Law No. 6698 on the Protection of Personal Data (KVKK).

- **Data minimization and explicit consent:** the system collects only the data "strictly necessary" for calculating the learning path (target university, test results, sleep restrictions). Upon first logging into the platform, texts regarding explicit consent are displayed to the user.
- **Cryptography:** As tested and confirmed in Section 9 (TC-10), personal identifiable information and user passwords are not stored in plain text in the database, but are irreversibly secured using AES-256 encryption and SHA-256 hash algorithms.

### 11.2 Ethical Implications

- **AI Bias and "Human-in-the-Loop":** The risk that a fully autonomous AI in the field of education might misjudge a student's abilities and thus design demotivating learning paths constitutes an ethical problem. To eliminate this risk, ADEXOS has added a "teacher" validation mechanism to the system. Decisions are not left directly to the machine but are subject to pedagogical oversight.
- **Mental Health and Stress Management:** When a student fails to complete their tasks, the system does not penalize them or stress them out with red warnings, but

recalculates the learning path using a flexible and empathetic approach (70% re-routing rule). This is an ethical design choice that protects the student's mental health.

### 11.3 UN Sustainable Development Goals

The platform offers a technological solution for two fundamental goals of the United Nations Sustainable Development Goals (SDGs):

- **SDG 4 – Quality Education:** for students who do not have access to expensive private lessons or private tutors, the platform uses advanced AI algorithms to create a personalized "educational coaching" service available 24 hours a day, which promotes equal opportunity and improves the quality of education.
- **SDG 10 – Reduced Inequalities:** by enabling any high school student with internet access—regardless of where they live or their economic situation—to benefit from high-level analyses integrated with data from the YÖK Atlas, it helps reduce the socioeconomic gap in the education system.

## SECTION 12: CONCLUSION

### 12.1 Summary of Accomplishments

As part of the ADEXOS project, we developed a smart educational platform with a hybrid architecture (N-tier microservices in C# and Python) that provides a radical solution to the problems of "data tracking" and "scheduling" when preparing students for the university entrance exam (YKS). The database of incorrect answers based on optical character recognition (OCR), the heat map, autonomous route recalculation (rerouting), and dynamic weekly planning functions approved by teachers were successfully modeled; we fully documented the project's system design, interface mockups, database architecture, and four-stage software test plan.

### 12.2 Key Challenges Encountered

The greatest technical challenge encountered during the project was running computationally intensive machine learning processes (Python) in sync with the main system's orchestration (C#). The bottleneck that would have resulted from attempting to combine two different ecosystems within a monolithic structure was identified, and a "microservices" architecture was adopted as a solution. Communication was handled via REST protocols based on gRPC and JSON. The second major challenge concerned the high hardware costs of the OCR model; these were circumvented by integrating the external Google Cloud Vision API service rather than retraining the model from scratch.

### 12.3 Individual Reflections

**Ahmet Can Dedeagili:** Serving as the software architect on a two-person team for this project was an incredibly rewarding experience for me. Defining the project's scope from the outset, conducting stakeholder analyses, and breaking down a complex software development process into distinct phases to plan the effort, time required, and risks allowed me, as an engineer, to develop the ability to see the big picture. On the technical side, my greatest motivation was designing the hybrid architecture at the heart of ADEXOS. Establishing communication between the central C#-based backend system and the Python-based AI microservice, securely ensuring integrations with external APIs, and modeling this entire structure according to object-oriented design principles took my technical skills to a whole new level. By supporting the resulting code architecture with modern CI/CD processes and a robust technology stack, we developed the project's deployment strategy. Ensuring compliance with data protection regulations and the system's ethical boundaries taught me that it's not just about writing code, but about developing a product that benefits society and adheres to the law.

**Mustafa Can :** Working on the ADEXOS platform was one of the most challenging yet rewarding experiences of my academic career. Before starting the project, it seemed overwhelming to me, but with careful planning, it turned into an activity I truly enjoyed. Having to work with just two people instead of the originally planned four pushed us to quickly take on cross-functional responsibilities and push our technical limits. Throughout this process, I found myself right at the heart of the system's core architecture: ensuring seamless communication between the C# ASP.NET backend, the Python AI microservice, and external APIs—without compromising on performance or security—became my top priority. The most important lesson I took away from this project was just how critical meticulous software engineering methodologies are. Ultimately, this intensive project bridged the gap between theoretical software engineering concepts and real-world system design, proving to me that clear documentation, risk management, and modular architecture are just as crucial as the code itself.

**SECTION 13: References**

**[1]** R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2014.

**[2]** F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*, Anniversary ed. Boston, MA, USA: Addison-Wesley Professional, 1995.

**[3]** I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.

**[4]** IEEE Computer Society, "IEEE Recommended Practice for Software Requirements Specifications," *IEEE Std 830-1998*, pp. 1-40, 1998.

**[5]** P. Brusilovsky and C. Peylo, "Adaptive and Intelligent Web-based Educational Systems," *International Journal of Artificial Intelligence in Education*, vol. 13, pp. 156-169, 2003.

**[6]** K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, Scrum.org, 2020.

**[7]** A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

**[8]** S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: defining gamification," in *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, 2011, pp. 9-15.

**[9]** S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

# APPENDIX A: INDIVIDUAL CONTRIBUTION LOG

**Member 1: Ahmet Can Dedeağılı**

- **Full Name and Student ID:** Ahmet Can Dedeağılı – 2211504005
- **Role(s) Assumed:** Project Manager, Software Architect, Core UML Designer.
- **Specific Sections Authored:**
    - Section 2: Project Scope & Stakeholder Analysis
    - Section 4: Software Project Management Plan (SPMP)
    - Section 5: Risk Management
    - Section 7: Software Design (Sub-sections 7.1, 7.2, 7.3, 7.4)
    - Section 8: Implementation Plan
    - Section 11: Ethical, Legal, and Sustainability Considerations
    - Section 12: Conclusion
- **Specific Diagrams Created:**
    - Stakeholder Interest/Influence Matrix (2.3)
    - Activity-on-Node (AoN) Network Diagram (4.3.1)
    - Gantt Chart (4.3.2)
    - Risk Matrix (5.2)
    - Class Diagram (7.2)
    - Sequence Diagrams (7.3)
    - Activity Diagram (7.4)
- **Estimated Number of Hours Contributed:** 70 Hours
- **Personal Reflection:** Serving as the software architect on a two-person team for this project was an incredibly rewarding experience for me. Defining the project's scope from the outset, conducting stakeholder analyses, and breaking down a complex software development process into distinct phases to plan the effort, time required, and risks allowed me, as an engineer, to develop the ability to see the big picture. On the technical side, my greatest motivation was designing the hybrid architecture at the heart of ADEXOS. Establishing communication between the central C#-based backend system and the Python-based AI microservice, securely ensuring integrations with external APIs, and modeling this entire structure according to object-oriented design principles took my technical skills to a whole new level. By supporting the resulting code architecture with modern CI/CD processes and a robust technology stack, we developed the project's deployment strategy. Ensuring compliance with data

protection regulations and the system's ethical boundaries taught me that it's not just about writing code, but about developing a product that benefits society and adheres to the law.

**Member 2: Mustafa Can**

- **Full Name and Student ID:** Mustafa Can – 2211504074
- **Role(s) Assumed:** Requirements Analyst, Database Designer, QA/Testing Lead.
- **Specific Sections Authored:**
    - Section 2 **:** Project Scope & Stakeholder Analysis
    - Section 3: Software Process Model
    - Section 6: Requirements Analysis
    - Section 7: Software Design (Sub-sections 7.5, 7.6, 7.7, 7.8)
    - Section 9: Testing Plan
    - Section 10: Maintenance Plan
- **Specific Diagrams Created:**
    - ADEXOS Agile Sprint Cycle (3.2)
    - Use Case Diagram (6.3)
    - State Machine Diagram (7.5)
    - Component Diagram (7.6)
    - Deployment Diagram (7.7)
    - Entity-Relationship (ER) Diagram (7.8)
    - Version Numbers Schema (10.3)
- **Estimated Number of Hours Contributed:** 70 Hours
- **Personal Reflection:** Working on the ADEXOS platform was one of the most challenging yet rewarding experiences of my academic career. Before starting the project, it seemed overwhelming to me, but with careful planning, it turned into an activity I truly enjoyed. Having to work with just two people instead of the originally planned four pushed us to quickly take on cross-functional responsibilities and push our technical limits. Throughout this process, I found myself right at the heart of the system's core architecture: ensuring seamless communication between the C# ASP.NET backend, the Python AI microservice, and external APIs—without compromising on performance or security—became my top priority. The most important lesson I took away from this project was just how critical meticulous

software engineering methodologies are. Ultimately, this intensive project bridged the gap between theoretical software engineering concepts and real-world system design, proving to me that clear documentation, risk management, and modular architecture are just as crucial as the code itself.